

Section 1 - Analysis

Introduction

For our Mini NEA project, we have been tasked with developing and programming a game for a handheld gamer which is modelled after the original nintendo Game & Watch - the succesor to the game boy. The model consists of a Raspberry Pi Pico with an attached Pico GFX board and may be coded using python.

About the Nintendo Game & Watch



The Original Gaming Watch!

The Game & Watch series is a collection of hand held game consoles developed by nintendo and first released in 1980. The original version of the gamer had a black and white LCD display and distinct 80s retro styling. The name 'Game & Watch' alludes to the device's capability to both display the time and play a singular game (depending on the model of the device). Since then there has been many generations of the Game & Watch with the most recent 40th anniversary edition being released in 2020 with a color display, a rechargeable battery and the ability to play multiple games.

StakeHolders and Users for my product

To help me get started on my project, and move forward with the analysis section, I needed to look at the potential stakeholders interacting with my product. For example, my primary user, Mr Wilde, who will be the person that I work most closely with when designing the product and ultimately who I am designing the game for.

Mr Wilde is a retro game enthusiast who enjoys playing and collecting games. He has a deep passion and interest in the history and progression of game consoles including the Game & Watch series. Some of his other interests include karate kid/cobra kai and lego.

Other prospective users include many KS3 students who regularly attend the Fabricarium club. They have a new found interest in gaming and the Game & Watch model may have a strong appeal towards them.

Analysis of previous Game & Watch games

- **Ball** - There are many popular games that were brought out on the Game & Watch including 'Ball' which was the first game released on the console and became a large hit for nintendo at the time. The game

idea was to have 2 balls that the player would toss up in the air and then catch and toss them up again like when someone is juggling. A point is scored with each successful catch and if a ball is dropped then the game is stopped. What I think made the game appealing to the crowd was the simplicity of the game yet the difficulty of it to progress further. Additionally, the wow-factor and functionality of the game dominated the previously empty market at the time.

Pros	Cons
First Game & Watch game	Limited controls combined with a low frame rate make game feel static
Game has a good difficulty progression curve as it gets harder to catch the balls overtime	Gameplay is very basic
Easy to understand due simplicity of idea	Lack of customization

- **Snoopy Tennis** - This game has 2 characters, charlie brown and snoopy. The user controls snoopy who is holding a tennis racket, and the user's goal is to hit all the tennis balls back that are launched towards him. The balls are thrown randomly between 3 levels, and snoopy can return these balls by moving between 3 levels of a tree and then swinging his racket. This is a fun and engaging game as the rate at which balls are launched increases over the duration of the game.

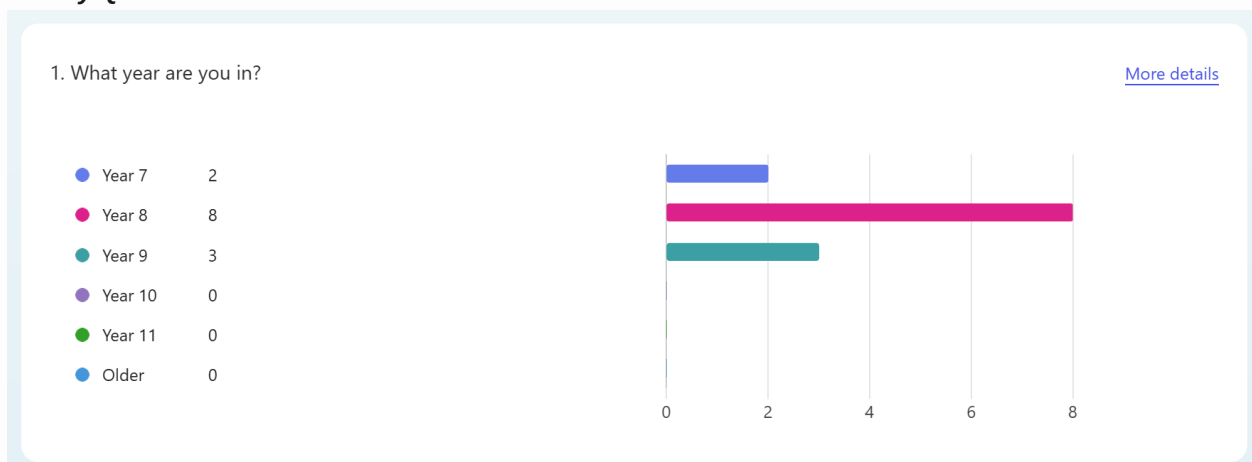
Pros	Cons
Uses famous characters from the comic strip 'Peanuts'	Lack of strategy in game may make it feel simple yet frustrating
Game quickly gets faster surprisingly quickly	Only 4 specific positions on tree
Randomness keeps player engaged	Gameplay soon becomes repetitive

Overall, I think both games share the same major con. Overtime, the game may become repetitive and feel exhaustive to play. They may feel underwhelming, especially when compared to more complex and interactive games on nintendo's newest hand held gaming model - the nintendo switch oled.

Survey Analysis

In order to collect more data around what prospective users are looking for in a game, we created a microsoft form and sent it out to the pupils attending Fabricarium. We totaled in 13 responses, and here is my interpretation of the results.

Survey Question 1



The above data shows that a part of our target demographic is in the younger age range from year 7 - year 9.

Survey Question 2

2. Rank the most important features of a game?

[More details](#)



The data above demonstrates what features or aspects of a game is important to my prospective users. However, I hope to implement some version of all listed features in my final game.

Survey Question 3

3. Rank the following game genres you want to play the most?

[More details](#)



Platform games, Action and Adventure games and Shooter games were ranked highest, which are all closely related and perhaps I could generate a game idea that merges together multiple genres to create a unique and popular product.

Survey Question 4

4. Is there other features i.e. Characters you want to see?

13 Responses

ID ↓	Name	Responses
13	Anish Dhanak	Have a unique style of graphics to create a different feeling for the player
12	Arnav Raghavendra	maunio instead of mario
11	Maun Vyas	MAUNIO instead of mario
10	Ajmain Mallik	unsure
9	Sammi Shum	KIRBY and YOSHI
8	Aaron Cheung	characters and lore behind Undertale/Deltarune

Survey Q4

A key response in this question was the highlighted emphasis on graphics and how the game looks, with a person suggesting to create your own style and looks. Another person, mentioned the games Undertale and Deltarune, which are indie RPGs created by Toby Fox. These games feature combat and an overall arching story line following a narrative.

Survey Question 5

5. Is there any specific (retro) games you want recreated?

13 Responses

ID	Name	Responses ↑
8	Aaron Cheung	1942
11	Maun Vyas	MARIO 64
5	Hao Xuan Guo	Mario
12	Arnav Raghavendra	Mario 64
4	Demian Freitag	Mario/Portal (1-2)
13	Anish Dhanak	Pac Man
10	Ajmain Mallik	Pokemon Red / Pokemon Blue

Survey Q5

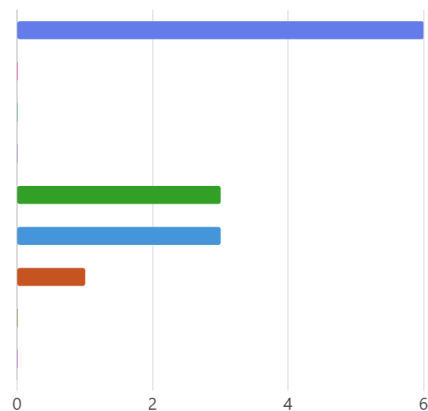
- Super Mario 64, a 3D platformer with innovative movement, procedural level design and the first ever 3D Mario game.
- Pac Man, an action maze chase game where you control pacman as ghosts try to hunt you
- 1942, a 2D flying game, similar to asteriods, but with fighter jets

Survey Question 6

6. What colour backlight would you want

[More details](#)

Black	6
White	0
Yellow	0
Orange	0
Red	3
Blue	3
Green	1
Pink	0
Purple	0



On a pico GFX board it is possible to change the backlight of the screen. This may be changed through the game, like turn red whenever the player takes damage. These responses indicate however that the majority would like the default backlight color to be plain.

Testing of Previous Mini NEA projects

We were provided with previous Mini NEA programs to give us an idea of the capability of the raspberry pi and pico GFX board. I decided to analyse the 2D platformer game.

Note:

```
from gfx_pack import GfxPack, SWITCH_A, SWITCH_B, SWITCH_C, SWITCH_D, SWITCH_E
```

Programs use this line to address the GfxPack library that is used in controlling the display

2D platformer:

This game has simple movement mechanics including left, right and jump, combined with collision detection and functional gravity simulation. Here is a look at some snippets of the python code:

```
# Player properties
self.player = {
    "x": 20,      # Starting X position
    "y": 30,      # Starting Y position
    "width": 4,   # Player width
    "height": 6,  # Player height
    "x_speed": 0, # Horizontal speed
    "y_speed": 0, # Vertical speed (for gravity)
    "is_jumping": False # Track if player is in the air
}

# Physics settings
self.GRAVITY = 0.5 # Gravity strength
self.JUMP_POWER = -4 # Jump strength (negative because y increases downward)
self.MOVE_SPEED = 2 # Horizontal movement speed

# Platform list - each platform is [x, y, width, height]
self.platforms = [
    [0, 50, 30, 4], # Starting platform
    [40, 40, 30, 4], # Middle platform
    [80, 30, 30, 4], # High platform
    [0, 60, 128, 4] # Ground
]
```

This above demonstrates the attributes set in the init function of the PlatformGame class. The function that is commonly used in all object orientated python programs to assign values to attributes on creation of a class. The comments explain the use of each attribute, which are being introduced into sub-categories including player properties (as a dictionary), physics settings and platforms.

```
def update_player(self):
    """Update player position and apply physics"""
    # Apply gravity
    self.player["y_speed"] += self.GRAVITY

    # Update position based on speed
    self.player["x"] += self.player["x_speed"]
    self.player["y"] += self.player["y_speed"]

    # Keep player within screen bounds
    if self.player["x"] < 0:
        self.player["x"] = 0
    elif self.player["x"] > self.WIDTH - self.player["width"]:
        self.player["x"] = self.WIDTH - self.player["width"]

    # Check if player fell off the bottom
    if self.player["y"] > self.HEIGHT:
        self.game_over = True
```

This is another important code segment that handles updating the player's position. Everytime the function is called, gravity is applied, the player position is updated based on the horizontal and vertical speed values. Than the players position is compared with hthe width and height of the display, in order to contain the player with the screen bounds. If the y position of the player is greater than the height of the screen, this mean the payer has fallen through the platforms and the game should end.

```

def check_platform_collisions(self):
    """Check and handle collisions between player and platforms"""
    self.player["is_jumping"] = True # Assume falling unless proven otherwise

    for platform in self.platforms:
        # Get platform boundaries
        plat_left = platform[0]
        plat_right = platform[0] + platform[2]
        plat_top = platform[1]
        plat_bottom = platform[1] + platform[3]

        # Get player boundaries
        player_left = self.player["x"]
        player_right = self.player["x"] + self.player["width"]
        player_top = self.player["y"]
        player_bottom = self.player["y"] + self.player["height"]

        # Check for collision
        if (player_right > plat_left and
            player_left < plat_right and
            player_bottom > plat_top and
            player_top < plat_bottom):

            # If player is falling and above platform
            if self.player["y_speed"] > 0 and player_top < plat_top:
                self.player["y"] = plat_top - self.player["height"]
                self.player["y_speed"] = 0
                self.player["is_jumping"] = False

```

The function above takes care of collision handling. It loops through each platform, then sets each platform boundaries based on the platform positions. Then it gets the current boundaries of the player based on its position. Finally, it compares each boundary of the player with the appropriate boundary of the platform, and adjusts the player properties if it lands on top of the platform.

Pros	Cons
Basic movement mechanics - left/right/jump	Movement is not smooth and there is no acceleration making it feel unnatural
Game can be easily reset with a click of a button	Collisions are only detected when the player lands on the top of the platform
Program is written and structured well	Player is restricted in small environment

Game Idea/ Design Brief

Game name: **PIXEL RAMPAGE** - this name I think best represents the art style and gameplay, while sounding catchy.

Design Brief: I will develop an infinite 2D platform shooter game. The scoring system will be based upon the number of 'coins' collected, the number of enemies killed and how far you have travelled/survived in the game. The graphic theme/visual style of the game will be pixel art inspired with an aim of keeping sprites simple to understand, and make the best use of the limited resolution of the display. Enemies should also be able to move and shoot in order to retaliate against the player and keep them engaged.

Specification Points

Here is a list of the minimum expectations set up for me to complete as I develop the game:

Objective	Explanation
-----------	-------------

Objective	Explanation
Movement	The character should have the ability to move left, right and jump. Complex movement mechanics may be added such as double jump or crouch however these are not essential
Shooting	The player should be able to shoot projectiles from a gun held by the character based on the input from the user. These projectiles should be able to correctly interact with surrounding objects and take damage on collisions with targets
Scoring	The game should be able to count and display the score of a session in the top corner of the display, updating the score based on a set of fixed rules in the game
Health	The player's current health should be displayed via a health bar and update appropriately. The player may take damage from enemy projectiles aswell as heal from pickups or even slowly overtime
Pickups	The character should be able to pick up coins/hearts/weapons and other in game items from on the platforms, which may also contribute to the score of the player or effect there gameplay capability
Procedural generation of levels	The game should theoretically be able to continue forever (until the player dies), as the level is randomly extended using platforms of different widths, heights, positions and number of coins on the platform
Enemies	There should be many different levels of enemies that are randomly spawned in on platforms and that can move and shoot at the player based upons its position
Animations	Some objects should be animated in order to make the game more lively and engaging. For example there should be animations in the character movement, and the movement of the projectiles
Restart function	The user should be able to restart the game whenever needed
User interface	There should be decorated pop up panels such as a main menu and game over page, directing the player and allowing them to seamlessly transision from one to another and start the game
Ease of Use	A 'help' pop up panel designed to explain how to play the game and make it easier to understand. This may help users first learn the controls of the game when starting out. This pop up may be static or interactive

Objective	Explanation
Leaderboard	The top 10 highest scores should be locally stored and saved on a text file on the device. The leaderboard should be able to be viewed and updated from within the game

Acceptable Limitations

These refer to restrictions on the functionality of the game usually caused by hardware capability as explained in the following cases

- Monochromatic Display - since the colour of each pixel can not be controlled due to the limitations of the gfx display, characters and sprites must be constructed from on and off pixels. However, you are able to change the colour of the backlight of the display allowing for you to change the overall game feel, i.e. backlight turns red when low health in order to overcome this problem
- Username next to score on leaderboard - due to the limited number of buttons on the model game, there is no effective way for the player to type in their name in so that it can be stored on the leaderboard
- Lack of accurate detail in animations - the low resolution and blur of the screen may make some animations seem unrecognisable when played at the correct speed and in movement. Therefore, I will aim to keep them basic and only on the player
- Enemy AI - due to the limiting processing power of the pico, I will write a rule based/scripted AI system with pre programmed behaviour, rather than an adaptive model. This is also more suitable for the more simple nature of the game

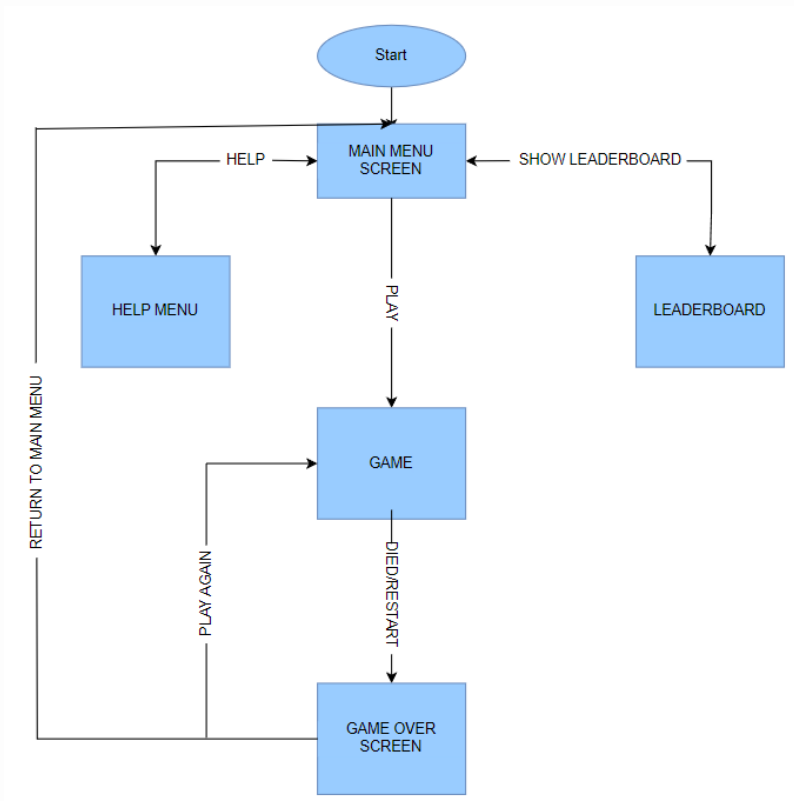
Section 2 - Documented Design

Introduction

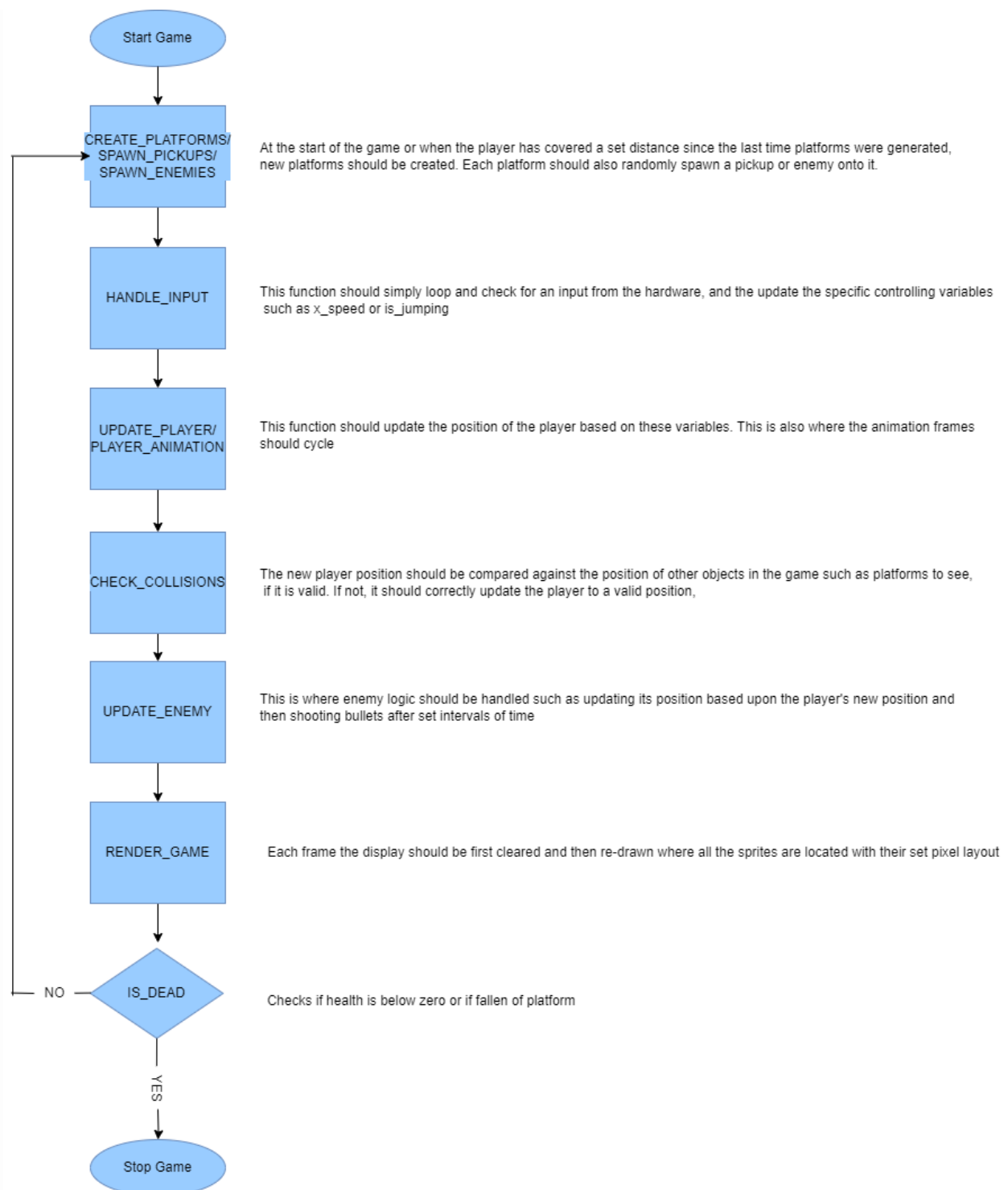
During the development stages for our Mini NEA project we have been tasked in recording our design processes and effectively documenting our thinking. In this section I will include explanations and images as I work towards the complete technical solution

Modeling the Problem

I first wanted to roughly break down/decompose the interface for the game and functions within the game itself. To this I created general abstracted flowcharts with the basic order of operations in the program.



General interface flowchart between menus



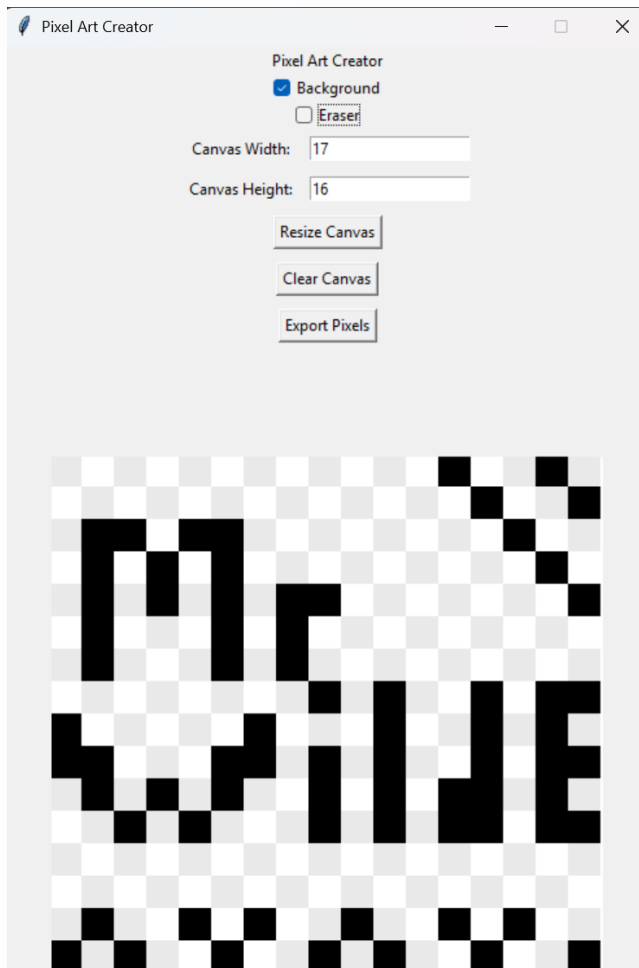
Game controlling and logic flowchart

Pixel Art Creator

In order to create some game art for pixel rampage, I needed to choose a suitable data structure for holding data for sprites in the game, so that I can easily draw and display them. The simplest way I thought of would be a binary 2D array where 1 represents black (pixel is drawn) and 0 represents white (pixels is not drawn). There will be no local position attached to each pixel. Just a starting position that represent where the top left corner should be drawn.

To implement this idea, I started by created a basic app/GUI that allows the user to draw shapes onto a canvas and export them in this format. Additionally, it allows the user to specifically resize their canvas to their needs, erase their mistakes and has a grey and white checkerboard pattern to improve the ease of use and visibility of each pixel.

Here is a link to the full program:[Pixel Art Creator](#)



PIXELS: [[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1], [0, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0], [0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0], [0, 1, 0, 1, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1], [0, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 1, 1], [1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0], [1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1], [0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0], [0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 1], [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0], [1, 0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 1]]

Below is the full commented on program in code snippets, demonstrating how I created this small application, using the tkinter library in python:

Importing libraries and initializing variables to default values:

```

import tkinter as tk
from tkinter import ttk
from tkinter import *
#Importing all libraries

root = tk.Tk()
root.geometry("500x800") #Defining the size of the window
root.resizable(0, 0) #Setting the window to remain at this fixed geometry
root.title("Pixel Art Creator")

#Size of canvas
CANVAS_WIDTH = 400
CANVAS_HEIGHT = 400

#Number of pixels on the canvas
XPixels = 16
YPixels = 16

#The fixed width and height of each of these pixels. (16 x 25 = 400)
WIDTH = 25
HEIGHT = 25

#Creates an empty version of the final array to be exported
Array = [[([0] * XPixels) for _ in range(YPixels)] #16x16 is the default size]

Background = False

Eraser = tk.StringVar(value="False") #Initial value False

```

The create canvas will create a blank white canvas with the correct width and Height. The draw function handles creating squares on the canvas and updating the binary 2D array to store the information. ExportPixels simply prints the array on command.

```

def CreateCanvas():
    global canvas

    canvas = tk.Canvas(root, width=CANVAS_WIDTH, height=CANVAS_HEIGHT, bg="white") #Creates the canvas with the set widthxheight
    canvas.pack(anchor=tk.CENTER, expand=True)

    # Re-bind events here
    canvas.bind("<Button-1>", draw) #Binds the left mouse button
    canvas.bind("<B1-Motion>", draw) #Bind the motion of the cursor

def draw(event):
    global Background
    global Array
    XCursorPos, YCursorPos = event.x, event.y #Gets cursor position on canvas
    #Identifies which pixel and the location of it on the canvas
    Xpos = (XCursorPos // WIDTH) * WIDTH
    Ypos = (YCursorPos // HEIGHT) * HEIGHT
    if 0 <= Xpos <= CANVAS_WIDTH and 0 <= Ypos <= CANVAS_HEIGHT and 0 <= Xpos + WIDTH <= CANVAS_WIDTH and 0 <= Ypos + HEIGHT <= CANVAS_HEIGHT: #Check against bounds of canvas
        if Eraser.get() == "False":
            canvas.create_rectangle((Xpos, Ypos), (Xpos + WIDTH, Ypos + HEIGHT), fill="black", outline = "") #Draws rectangle as pixel in position
            Array[YCursorPos // HEIGHT][XCursorPos // WIDTH] = 1 #Updates the array
        else:
            #ERASER
            if Array[YCursorPos // HEIGHT][XCursorPos // WIDTH] == 1: #If the pixel is black
                canvas.create_rectangle((Xpos, Ypos), (Xpos + WIDTH, Ypos + HEIGHT), fill="white", outline = "") #Draw white rectangle
                Array[YCursorPos // HEIGHT][XCursorPos // WIDTH] = 0 #Updates the array
                Background = not Background
                BackgroundChanged() #Re-call background to account for adjustment, i.e. the erased square may be grey or white on checkerboard

def ExportPixels():
    print("Exporting Pixels")
    print(Array) #Outputs array

```

These are the functions binded to the buttons for additional ease of use features such as toggling the background, resizing the canvas and clearing it.

BUTTON FUNCTIONS

```
def BackgroundChanged(clear=False):
    global Background
    Background = not Background # the new background should be the opposite of the current background setting
    Grey1 = False
    Grey2 = False
    for y in range(YPixels):
        Grey1 = not Grey2 # Top left hand pixel will be grey
        Grey2 = not Grey2 # Alternate Grey2 by the column so that each row starts grey...white...grey etc
        for x in range(XPixels):
            if Grey1:
                if Background:
                    colour = "#E8E9E8" #grey
                else:
                    colour = "white" #white
            if Array[y][x] == 0 or clear:
                canvas.create_rectangle((x*WIDTH, y*HEIGHT), ((x*WIDTH) + WIDTH, (y*HEIGHT) + HEIGHT), fill=colour,
                outline="")
            Grey1 = not Grey1 # Alternates Grey1 by each square

def ResizeCanvas():
    #Global variable needed to create new canvas
    global Array
    global canvas
    global XPixels
    global YPixels
    global CANVAS_WIDTH
    global CANVAS_HEIGHT
    ClearCanvas()
    print("Resizing Canvas")

    canvas.destroy() #Destroys current canvas
    XPixels = int(canvasWidthEntry.get())
    YPixels = int(canvasHeightEntry.get())

    #New canvas width and hight
    CANVAS_WIDTH = XPixels * WIDTH
    CANVAS_HEIGHT = YPixels * HEIGHT

    CreateCanvas()
    Array = [[[0] * XPixels) for _ in range(YPixels)] #New empty array

def ClearCanvas():
    global Array
    global Background
    canvas.delete("all") #Deletes all rectangles on the canvas
    print("Clearing Canvas")
    Array = [[[0] * XPixels) for _ in range(YPixels)] #New empty array
    Background = not Background #Re-call background to account for adjustment
    BackgroundChanged(True)
```

This segment setups all the UI buttons and input fields:

```

title = tk.Label(root, text="Pixel Art Creator")
title.pack()

#Checkboxes - Background and Eraser
ttk.Checkbutton(root,
    text='Background',
    command=BackgroundChanged).pack()

ttk.Checkbutton(root,
    text='Eraser',
    variable=Eraser,
    onvalue='True',
    offvalue='False').pack()

frame = tk.Frame(root)
frame.pack()

canvasWidth = tk.StringVar() #Use of string var to get entry data
canvasHeight = tk.StringVar()

canvasWidthLabel = tk.Label(frame, text="Canvas Width:")
canvasWidthEntry = tk.Entry(frame)

canvasHeightLabel = tk.Label(frame, text="Canvas Height:")
canvasHeightEntry = tk.Entry(frame)

#Labels and input fields arranged in grid format
canvasWidthLabel.grid(row=0, column=0, padx=5, pady=5)
canvasWidthEntry.grid(row=0, column=1, padx=5, pady=5)
canvasHeightLabel.grid(row=1, column=0, padx=5, pady=5)
canvasHeightEntry.grid(row=1, column=1, padx=5, pady=5)

#3 Buttons mapped to function
resizeCanvasButton = tk.Button(root, text="Resize Canvas", command=ResizeCanvas)
resizeCanvasButton.pack(pady=5)

clearCanvasButton = tk.Button(root, text="Clear Canvas", command=ClearCanvas)
clearCanvasButton.pack(pady=5)

exportPixelsButton = tk.Button(root, text="Export Pixels", command=ExportPixels)
exportPixelsButton.pack(pady=5)

CreateCanvas() #Canvas is at the bottom

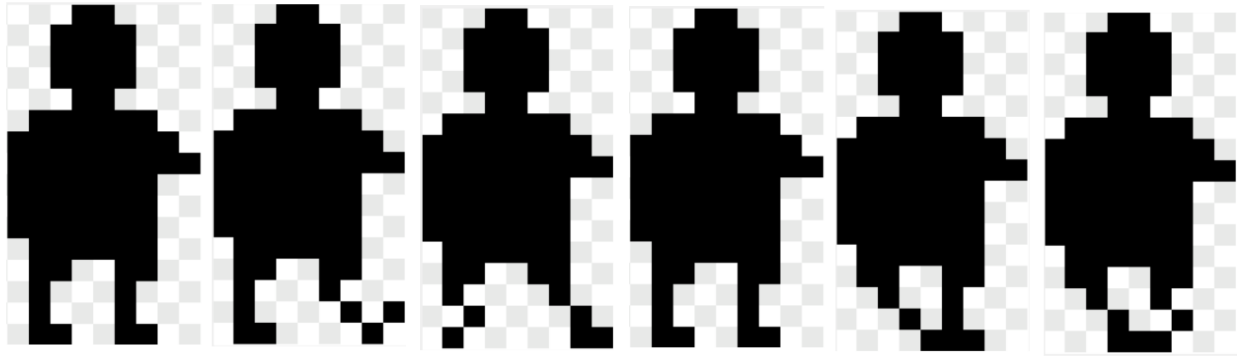
root.mainloop()

```

Creating the sprites

After looking at some pixel art photos online, I started designing the sprites for my game. I wanted to account for the size of the display (128x64) and create my sprites relative to that.

I wanted the player to remain front on to the camera so that the orientation would be clear. I animated just the legs, as I wanted the upperbody to be fixed with the weapon. This is also why the hand extends out slightly.



9x16 Player front gun walking animation

Below are my five different gun designs where each will also have different properties such as cooling time, range and damage.



Gun0 - revolver

Gun1 - ~~smg~~

Gun2 - large pistol

Gun3 - automatic

Gun4 - machine gun

Here are my pickups designs made to be easily recognisable. With clear additional scores and the empty heart signifying +50 health and the full heart signifying +100 health



11x7 +50 score

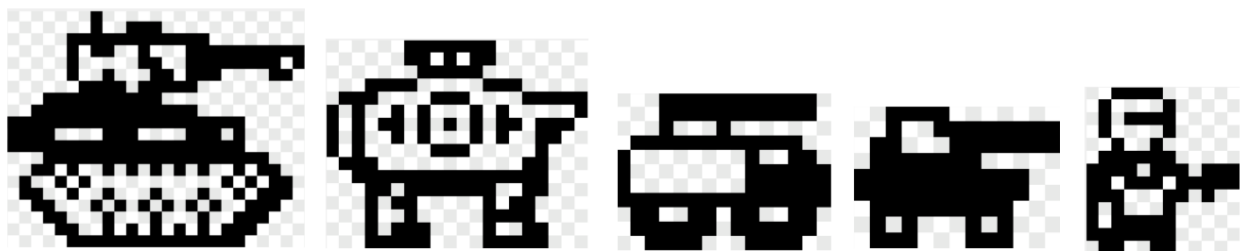
15x7 +100 Score

15x7 +500 Score

13x11 empty heart

13x11 full heart

Finally, here are my enemies ranging from biggest to smallest. Each has their built in attached gun with different properties and their should be different enemy properties like max health aswell. They will be able to move along platforms but not jump between them.



25x20 Big Tank

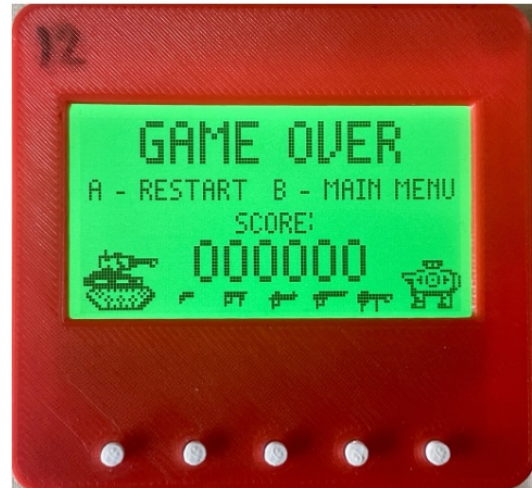
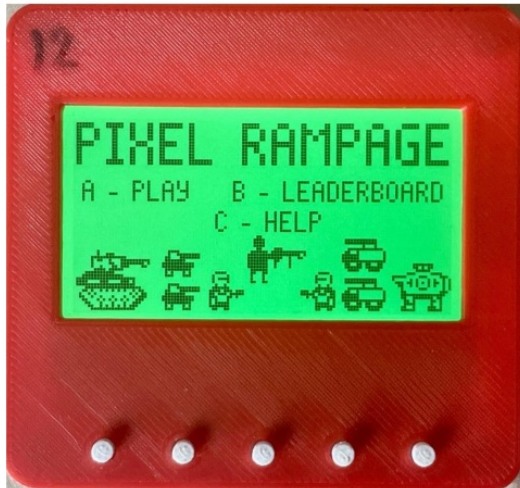
20x16 Robot

15x10 Truck

13x9 Small Moving Turret

12 x 13 Small Enemy

After making these sprites and then writing a function to display the array I was able to create these 2 decorated pop up panels:



Object properties and attributes

Player: - Pixels - the sprite to be displayed - Health - from 1 to 100 - Score - current score of player that will update as game goes on - currentWeapon - the weapon the player is using - Xpos - Ypos - Xspeed - Yspeed - width - height - slide amount - the rate at which the player will decelerate or slide - jump force - the initial speed the player will jump up with - gravity - the constant acceleration value downwards - direction - the current orientation of the player left or right - currentFrame - the current frame number in the animation cycle

Gun: - Pixels - the sprite to be displayed - weaponNum - unique number given to each gun - Range - how far the bullet can travel before being deleted - Max time between shots - Time since last shot - Damage - Barrel position - the local position relative to the gun where bullets should be spawned

Bullet: - Xpos - Ypos - Angle - Distance Travelled - how far the bullet has moved - Xspeed - Damage

Platform: - Xpos - Ypos - width - height - this should be fixed for every platform (global)

Pickup: - Pixels - the sprite to be displayed - pickupType - the type of pickup it is - Xpos - Ypos

Algorithms of Solution in Pseudocode

To get an idea of how I would design solutions to the complex parts of my game I first decided to use pseudocode to create the general order of instructions to the most important procedures such as drawing, player movement, shooting, and creating the level, without including the very specifics

DRAWING

```

DRAW(Pixels, StartXpos, StartYpos):
    CheckBounds()

    SetPen(White)
    ClearScreen()

    SetPen(Black)

    For y in range(len(pixels)):
        For x in range(len(pixels[y])):
            if pixels[y][x] == 1:
                DISPLAY.SetPixel(StartXpos + X, StartYpos + Y)
  
```

PLAYER MOVEMENT

```
HANDLE INPUT:
  if SWITCH_A:
    Xspeed = min(maxSpeed, Xspeed - accel)
  elif SWITCH_B:
    Xspeed = min(maxSpeed, Xspeed + accel)
  else:
    Xspeed = 0

  if SWITCH_C:
    Yspeed = min(maxSpeed, Yspeed - accel)
  elif SWITCH_D:
    Yspeed = min(maxSpeed, Yspeed + accel)
  else:
    Yspeed = 0

UPDATE PLAYER:
  Xpos += Xspeed
  Ypos += Yspeed
SHOOTING

bullets = []

SPAWN BULLETS(StartXpos, StartYpos, Speed):
  bullet = new Bullet(StartXpos, StartYpos, Speed)
  bullets.append(bullet)

DELETE BULLETS:
  For bullet in bullets:
    if bullet.isOverlap():
      bullets.pop(bullet)

UPDATE BULLETS:
  For bullet in bullets:
    bullet.Xpos += bullet.Speed
CREATING PLATFORMS

platforms = []
CREATE PLATFORMS(numberOfPlatforms):
  For x in range(numberOfPlatforms):
    platform = Platform(lastPlatform.Xpos + distanceBetweenPlatforms, randomInt(minHeight, maxHeight))
    platforms.append(platform)
  SPAWN PICKUPS()
  SPAWN ENEMIES()
LEADERBOARD

UPDATE LEADERBOARD(newScore):
  if newScore > lowestScoreOnLeaderBoard:
    leaderboard.pop()
    leaderboard.append(newScore)
    leaderboard.sort()
    print("Your on the leaderboard")
```

Input Process Storage Output Chart - IPSO

IPSO charts are used as a form of algorithm presentation. In this table I have gone iinto further than the pseudocode above

Algorithm	Input	Process	Storage	Output
	Pixels - Binary Array, StartXpos - top			

Algorithm	Input	Process	Storage	Output
Draw	left hand corner, StartYpos - top left hand corner, isFlipped, cameraViewPortXPos - What area should be rendered, displayWidth, displayHeight	Draws each pixel array onto the display by looping through the array	-	The sprite
PlayerMovement	Button Switches for controlling left,right etc	Updates Xspeed and Yspeed based upon uniform acceleration value and then the new XPos and Ypos. It should then check for collisions	-	New player movement values
Shooting	Button Switches - one button is allocated for this	Spawns new bullets after a given time if holding down button. Updates existing bullets and destroyed them if they have collided with another platform/enemy	-	Bullet list
CreatingPlatforms	currentDistance Covered - how far the player has travelled in the game, numberOfPlatforms - the number of additional platforms to be generated	Spawns new platforms at a random height	-	Platform List
Leaderboard	newScore - the player score to see if on leaderboard	compares the player's score against the leaderboard and then will reorder the leaderboard if needed	A text file that stores all the scores on leaderboard. This text file should be carefully formatted as their is limited storage space, and should be not volatile so that high scores are	New leaderboard

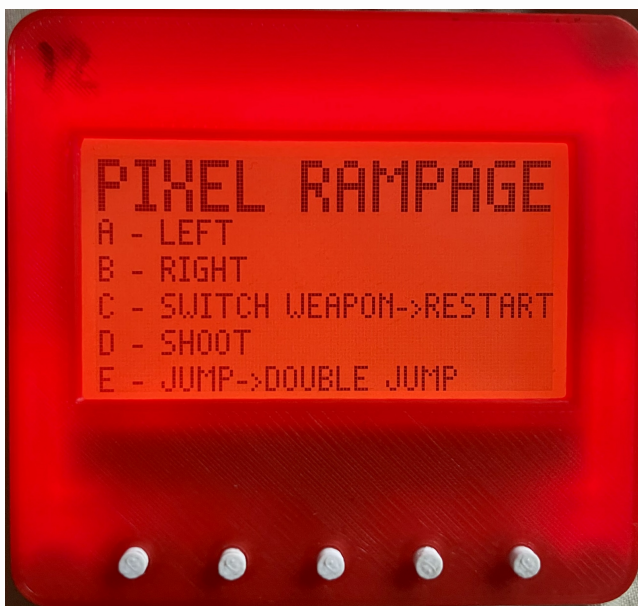
Algorithm	Input	Process	saved locally to Storage the device	Output

Player Controls

I will be referring to 5 switches on the device from A, B, C, D, and E from left to right. Now, I am assigning functions/commands to each button as I have a good understanding in what the gameplay will be like:

- SWITCH_A: Hold to accelerate and turn the player to the left
- SWITCH_B: Hold to accelerate and turn the player to the right
- SWITCH_C: Used to switch the player's weapon between picked up items. Hold the button to restart the game
- SWITCH_D: Hold to shoot bullets
- SWITCH_E: Press once to jump and twice to double jump

I made this as concise as possible in the static help menu as shown below:



Further Notes

- Player advantage - to give the player an upper hand over the enemies and as a bonus feature I decided it to make it so that the player will automatic shoot bullets at an angle to target the closest enemy, however enemies may only shoot laterally. This will allow to the player to dodge the enemy easily while still keeping target enhancing the game play. To do this I will need to follow the pseudocode algorithm below.

CALCULATE ANGLE(enemy):

dx = enemy.Xpos - playerXpos

dy = enemy.Ypos - playerYpos

return (ARCTAN(dy/dx))

CLOSEST ENEMY:

closestEnemy = None

minDistance = INFINITY

For enemy in enemies:

distance = | playerXpos - enemy.Xpos |

if distance < minDistance: (< not <= so if 2 enemies are next to each other than the first one spawned will be the closestEnemy)

minDistance = distance

closestEnemy = enemy

return CALCULATE ANGLE(closestEnemy)

SHOOT AT ANGLE:

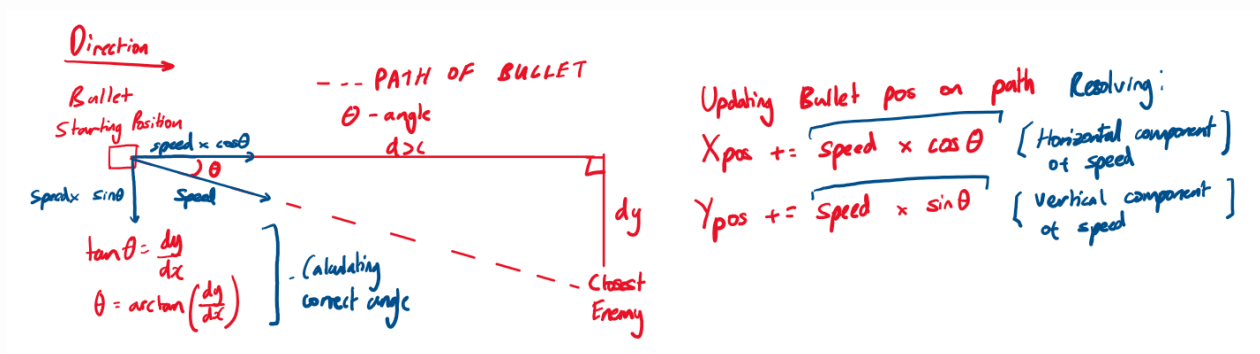
(assuming direction is right)

angle = CLOSEST ENEMY()

bullet.Xpos += bullet.speed * COS(bullet.angle)

bullet.Ypos += bullet.speed * SIN(bullet.angle)

The picture below demonstrates the mathematical ideas behind this solution:



- Camera viewport - as the player moves around the map different parts of the map and related object should be displayed. In order to control what is displayed, I need to you the following algorithm

cameraFollowDistance = 30 (some fixed value)

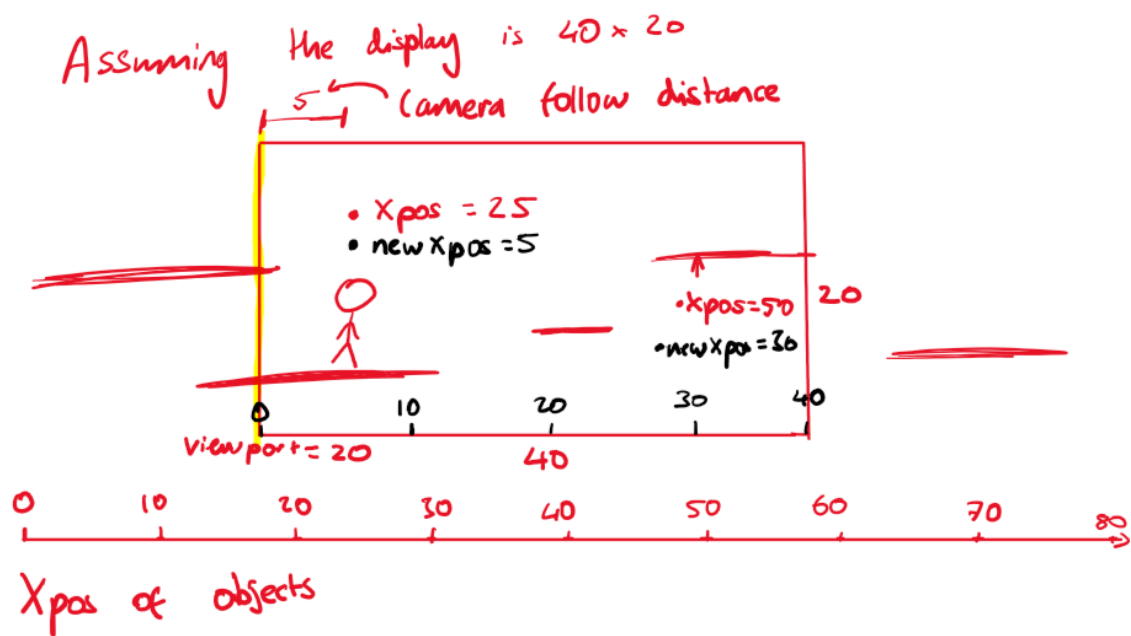
RENDER GAME:

viewPort = playerXpos - cameraFollowDistance

(Then when drawing an object we will pass in the following values)

DRAW(object.pixels, object.Xpos - viewPort, object.Ypos)

The picture below visually demonstrates the idea:



$$viewPort = X_{pos} - 5$$

Everything is then shifted to the left by this viewport amount

Section 3 - Technical Solution

Introduction

This is my final product where I have implemented everything stated in the previous sections. I will first demonstrate my game working and then clearly explain all the parts of my program.

PIXEL RAMPAGE Demo

This is a video link to me playing the game showing many of the features and gameplay [Pixel Rampage Demo](#) (camera quality is limited)

Final Product Link

[THE CODE](#)

Program Analysis

I have separated the program into different code snippets and please carefully read the additional comments on the program aswell.

Importing the gfx_pack library (documentation here - [pimoroni documentation](#)) which allows me to interact with the display. I have also imported time, random and math as I have functions like sleep and randint within other parts of the program

```
from gfx_pack import GfxPack, SWITCH_A, SWITCH_B, SWITCH_C, SWITCH_D, SWITCH_E
import time
import random
import math
```

I created an overall game class that co-ordinates everything, and then will have sub-classes within it such as Enemies and Guns. Each sub-class includes all of the properties I discussed previously in the documented design stages, and I am simply assigning values passed into the constructor (init) method, on creation to be used later specific to an instance of the object. Notably, the bullet class includes an update function, which is continously called when the bullet is not destroyed. It works by moving the bullet along by the correct horizontal/vertical direction and distance, so that each bullet travels along a fixed, set straight path through it's lifetime.

```

class Game:
    class Gun:
        def __init__(self, shootingSpeed, range, maxTimeBetweenShots, pixels, gunHolderLocalXpos, gunHolderLocalYpos,
        barrelLocalXpos, barrelLocalYpos, damage):
            #Assigning values on creation inside the object to be used later
            self.shootingSpeed = shootingSpeed
            self.range = range
            self.maxTimeBetweenShots = maxTimeBetweenShots
            self.pixels = pixels

            #GunHandling
            self.gunHolderLocalXpos = gunHolderLocalXpos #This marks the top left hand corner of the gun
            self.gunHolderLocalYpos = gunHolderLocalYpos
            self.barrelLocalXpos = barrelLocalXpos # This marks the position of the barrel where bullet are shot from
            self.barrelLocalYpos = barrelLocalYpos

            self.damage = damage

    class Pickup:
        def __init__(self, Xpos, Ypos, pickupTypeNum):
            self.Xpos = Xpos
            self.Ypos = Ypos
            self.pickupTypeNum = pickupTypeNum
            self.destroyed = False

    class Bullet:
        def __init__(self, Xpos, Ypos, shootingSpeed, range, isDirectionRight, damage, angle):
            self.Xpos = Xpos
            self.Ypos = Ypos
            self.distanceTravelled = 0
            self.shootingSpeed = shootingSpeed
            self.range = range
            self.destroyed = False
            self.isDirectionRight = isDirectionRight
            self.damage = damage
            self.angle = angle

        def update(self):
            if self.isDirectionRight:
                #Resolving speed into components using cos (horizontal) and sin (vertical) components
                #math uses radians when working with trig identities
                self.Xpos += self.shootingSpeed * math.cos(self.angle)
                self.Ypos += self.shootingSpeed * math.sin(self.angle)
            else:
                self.Xpos -= self.shootingSpeed * math.cos(self.angle)
                self.Ypos -= self.shootingSpeed * math.sin(self.angle)

            #Updates the total distance travelled
            self.distanceTravelled += self.shootingSpeed

            if self.distanceTravelled >= self.range:
                #When the bullet has travelled further than the range it is marked to be destroyed in the next frame
                self.destroyed = True

    class Platform:
        def __init__(self, Xpos, Ypos, width):
            self.heightOfPlatform = 2 #This marks it from the top going down (as y is positive)
            self.Xpos = Xpos
            self.Ypos = Ypos
            self.width = width
            #Height is fixed so doesn't needed to be stored in object

```

This is the definition of the enemy class which includes an update function as well where both shooting is called and the position is updated. Some of the enemies attributes are fixed and others vary based upon the

type of enemy such as health or their gun type. Additionally, the update function takes the Game class as a parameter so that it can call the enemy_shoot function and append to the existing bullets list allowing them to interact with other objects. *This segment of code works towards the **Enemies** objective:*

```
class Enemy:
    def __init__(self, enemyTypeNum, Xpos, Ypos, platform, weaponNum, enemyFireRate, enemyHealth):
        self.enemyTypeNum = enemyTypeNum #1-5
        self.enemyXpos = Xpos
        self.enemyYpos = Ypos
        self.detectionDistance = 100 #How far away the player will be before being in the 'range' of the enemy
        self.fireRandomness = 5 #Used to give some variance in enemy shooting
        self.platform = platform
        self.enemyfireRate = enemyFireRate
        self.enemyTimeSinceLastShot = 0
        self.enemyHealth = enemyHealth
        self.enemyDirectionRight = False
        self.closestDistanceToPlayer = 16 #The closest distance the enemy can get to the player. This will ensure the
        #player never 'camps' inside the enemy
        self.destroyed = False

    def update(self, playerXpos, GameScene):
        self.enemyDirectionRight = True if playerXpos - self.enemyXpos >= 0 else False #Identifies if the enemy should
        #be facing left or right to face player

        if self.enemyHealth <= 0:
            self.destroyed = True
            return

        #Fire
        if abs(self.enemyXpos - playerXpos) <= self.detectionDistance: #If player within range...
            if self.enemyTimeSinceLastShot > self.enemyfireRate:
                self.enemyTimeSinceLastShot = random.randint(-self.fireRandomness, self.fireRandomness) / 100 #reset
                #counter with number within range
                GameScene.enemy_shoot(self.enemyXpos, self.enemyYpos, self.enemyTypeNum, self.enemyDirectionRight)
            #shoot the bullet
            else:
                self.enemyTimeSinceLastShot += 0.01

        #Move
        multiplier = 1 if self.enemyDirectionRight == True else -1
        #Target position is as close to the player without exceeding the closestDistance or going of the platform
        self.targetPosition = max(self.platform.Xpos, min(playerXpos - multiplier *
        (len(GameScene.enemyTypes[self.enemyTypeNum][0]) + self.closestDistanceToPlayer), self.platform.Xpos +
        self.platform.width - len(GameScene.enemyTypes[self.enemyTypeNum][0])))
        self.enemyXpos = self.enemyXpos + ((self.targetPosition - self.enemyXpos) * 0.08) #Interpolate smoothly to target
        #position
```

This below segment sets the attributes of the sprites, movement properties and other game variables to their initial values. This init function is called on start up as well as when restarting the Game, to make sure that the player starts from scratch each time and progress resets completely. The function also stores all the pixel arrays exported from the pixel art creator application. I decided to use dictionaries to group certain attributes together under one title like seen in playerProperties and gunProperties.

```
def __init__(self):
    #Initilaze hardware
    self.gfx = GfxPack()
    self.display = self.gfx.display
    self.WIDTH, self.HEIGHT = self.display.get_bounds() #128x64

    #Set Font
    self.display.set_font("bitmap8")
```

```

#colours - as tuples that may be unpacked
self.Blue = (47,112,225, 1)
self.Orange = (255, 10, 0, 1)
self.Red = (255, 0, 0, 1)
self.Green = (60, 230, 5, 1)
self.defaultColour = self.Blue
self.gfx.set_backlight(*self.defaultColour)

#I used properties to divide up attributes of the game class
self.physicalProperties = {
    "Gravity" : 0.015
}

self.playerProperties = {
    "Xpos" : 30,      #Spawn Position, Xpos and Ypos mark the top left hand corner of the player
    "originalXpos" : 30,
    "Ypos" : 9,
    "colliderWidth" : 9,
    "colliderHeight" : 16,
    #MOVEMENT
    "Xspeed" : 0,
    "Yspeed" : 0,
    "accel" : 0.038,
    "minSpeed" : 0.2,
    "maxSpeed" : 1.8,
    "brakingForce" : 0.012,      #The lower this value, the easier it is to slide
    "jumpForce" : 0.38,
    "isJumping" : True,
    "readyToDoubleJump" : False,
    "hasJumped" : False,
    "isDoubleJumping" : True,
    "doubleJumpMultiplier" : 1.5,
    "isDirectionRight" : True,
    "canMoveX" : True,
    "maxHealth" : 100
}

self.playerFrame0 = [[0, 0, 0, 1, 1, 0, 0, 0, 0], [0, 0, 1, 1, 1, 1, 0, 0, 0], [0, 0, 1, 1, 1, 1, 0, 0, 0], [0, 0, 1, 1, 1, 1, 0, 0, 0],
[0, 0, 0, 1, 1, 0, 0, 0, 0], [0, 1, 1, 1, 1, 1, 1, 0, 0], [1, 1, 1, 1, 1, 1, 1, 1, 0], [1, 1, 1, 1, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 1, 1, 0, 0],
[1, 1, 1, 1, 1, 1, 1, 0, 0], [1, 1, 1, 1, 1, 1, 1, 0, 0], [0, 1, 1, 1, 1, 1, 1, 0, 0], [0, 1, 1, 0, 0, 1, 1, 0, 0], [0, 1, 0, 0, 0, 1, 0, 0, 0], [0,
1, 0, 0, 0, 1, 0, 0, 0], [0, 1, 1, 0, 0, 1, 1, 0, 0]]
self.playerFrame1 = [[0, 0, 0, 1, 1, 0, 0, 0, 0], [0, 0, 1, 1, 1, 1, 0, 0, 0], [0, 0, 1, 1, 1, 1, 0, 0, 0], [0, 0, 1, 1, 1, 1, 0, 0, 0],
[0, 0, 0, 1, 1, 0, 0, 0, 0], [0, 1, 1, 1, 1, 1, 1, 0, 0], [1, 1, 1, 1, 1, 1, 1, 1, 0], [1, 1, 1, 1, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 1, 1, 0, 0],
[1, 1, 1, 1, 1, 1, 1, 0, 0], [1, 1, 1, 1, 1, 1, 1, 0, 0], [0, 1, 1, 1, 1, 1, 1, 0, 0], [0, 1, 1, 0, 0, 1, 1, 0, 0], [0, 1, 0, 0, 0, 1, 0, 0, 0], [0,
1, 0, 0, 0, 1, 0, 1], [0, 1, 1, 0, 0, 0, 1, 0]]
self.playerFrame2 = [[0, 0, 0, 1, 1, 0, 0, 0, 0], [0, 0, 1, 1, 1, 1, 0, 0, 0], [0, 0, 1, 1, 1, 1, 0, 0, 0], [0, 0, 1, 1, 1, 1, 0, 0, 0],
[0, 0, 0, 1, 1, 0, 0, 0, 0], [0, 1, 1, 1, 1, 1, 1, 0, 0], [1, 1, 1, 1, 1, 1, 1, 1, 0], [1, 1, 1, 1, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 1, 1, 0, 0],
[1, 1, 1, 1, 1, 1, 1, 0, 0], [1, 1, 1, 1, 1, 1, 1, 0, 0], [0, 1, 1, 1, 1, 1, 1, 0, 0], [0, 1, 1, 0, 0, 1, 1, 0, 0], [0, 1, 0, 0, 0, 0, 1, 0, 0], [0,
0, 1, 0, 0, 0, 0, 1, 0], [0, 1, 0, 0, 0, 0, 0, 1, 1]]
self.playerFrame3 = [[0, 0, 0, 1, 1, 0, 0, 0, 0], [0, 0, 1, 1, 1, 1, 0, 0, 0], [0, 0, 1, 1, 1, 1, 0, 0, 0], [0, 0, 1, 1, 1, 1, 0, 0, 0],
[0, 0, 0, 1, 1, 0, 0, 0, 0], [0, 1, 1, 1, 1, 1, 1, 0, 0], [1, 1, 1, 1, 1, 1, 1, 1, 0], [1, 1, 1, 1, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 1, 1, 0, 0],
[1, 1, 1, 1, 1, 1, 1, 0, 0], [1, 1, 1, 1, 1, 1, 1, 0, 0], [0, 1, 1, 1, 1, 1, 1, 0, 0], [0, 1, 1, 0, 0, 1, 1, 0, 0], [0, 1, 0, 0, 0, 1, 0, 0, 0], [0,
1, 0, 0, 0, 1, 0, 0, 0], [0, 1, 1, 0, 0, 1, 1, 0, 0]]
self.playerFrame4 = [[0, 0, 0, 1, 1, 0, 0, 0, 0], [0, 0, 1, 1, 1, 1, 0, 0, 0], [0, 0, 1, 1, 1, 1, 0, 0, 0], [0, 0, 1, 1, 1, 1, 0, 0, 0],
[0, 0, 0, 1, 1, 0, 0, 0, 0], [0, 1, 1, 1, 1, 1, 1, 0, 0], [1, 1, 1, 1, 1, 1, 1, 1, 0], [1, 1, 1, 1, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 1, 1, 0, 0],
[1, 1, 1, 1, 1, 1, 1, 0, 0], [1, 1, 1, 1, 1, 1, 1, 0, 0], [0, 1, 1, 1, 1, 1, 1, 0, 0], [0, 1, 1, 0, 0, 1, 1, 0, 0], [0, 0, 1, 0, 0, 1, 0, 0, 0], [0,
0, 0, 1, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 1, 1, 0, 0]]
self.playerFrame5 = [[0, 0, 0, 1, 1, 0, 0, 0, 0], [0, 0, 1, 1, 1, 1, 0, 0, 0], [0, 0, 1, 1, 1, 1, 0, 0, 0], [0, 0, 1, 1, 1, 1, 0, 0, 0],
[0, 0, 0, 1, 1, 0, 0, 0, 0], [0, 1, 1, 1, 1, 1, 1, 0, 0], [1, 1, 1, 1, 1, 1, 1, 1, 0], [1, 1, 1, 1, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 1, 1, 0, 0],
[1, 1, 1, 1, 1, 1, 1, 0, 0], [1, 1, 1, 1, 1, 1, 1, 0, 0], [0, 1, 1, 1, 1, 1, 1, 0, 0], [0, 1, 1, 0, 0, 1, 1, 0, 0], [0, 0, 1, 0, 0, 1, 0, 0, 0], [0,
0, 0, 1, 0, 0, 1, 0, 0], [0, 0, 0, 1, 1, 1, 0, 0, 0]]

# 6 frames to cycle between
self.playerFrames = [self.playerFrame0, self.playerFrame1, self.playerFrame2, self.playerFrame3, self.playerFrame4,
self.playerFrame5]

```

self.playerName =

```
self.frameCounter = 0
self.maxFrameLength = 8
self.frameLength = self.maxFrameLength
self.frameNum = 0 #1-6

self.gunProperties = {
    #non-specific
    "isShooting" : False,
    "currentWeaponNum" : 0, #The current weapon is -1 when the character has no weapon, we will start with
the most basic weapon Gun0
    "timeSinceLastShot" : 0,
    "currentWeaponList" : [0] #Starts with Gun0
}

#GUNS
Gun0 = self.__class__.Gun(3.25, 105, 0.1, [[0, 1, 1, 1, 1], [1, 1, 1, 1, 1], [1, 1], [1]], 0, 5, 10, 5, 20)
Gun1 = self.__class__.Gun(2.58, 80, 0.2, [[1, 1, 1, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 1, 1, 0], [1, 0, 1, 0, 0, 1, 1, 0], [1, 1, 1,
0, 0, 1, 0, 0], [1, 0, 0, 0, 0, 1, 0, 0]], -1, 5, 12, 5, 25)
Gun2 = self.__class__.Gun(5.5, 115, 0.06, [[0, 0, 1, 1, 0, 0, 0, 0, 1, 0], [0, 1, 1, 1, 1, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 1, 1,
1, 1, 0], [0, 1, 0, 1, 0, 0, 0, 0, 0, 0], [0, 1, 1, 1, 0, 0, 0, 0, 0, 0], [0, 1, 0, 0, 0, 0, 0, 0, 0, 0]], -1, 4, 13, 5, 18)
Gun3 = self.__class__.Gun(3.4, 130, 0.08, [[0, 0, 1, 1, 1, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0], [1, 1, 1,
1, 1, 0, 0, 0, 0, 0, 0, 0], [0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0], [0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0], [0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]], -1,
4, 14, 4, 23)
Gun4 = self.__class__.Gun(5, 120, 0.11, [[0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0], [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1], [1, 1, 1, 1,
1, 1, 1, 1, 1, 0, 1], [1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0], [0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0], [0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0], [0, 1, 0,
0, 0, 1, 0, 0, 0, 0, 0, 0]], -1, 4, 14, 5, 35)
#GunX etc.
self.guns = [Gun0, Gun1, Gun2, Gun3, Gun4] #This order determines the weaponNum

#PICKUPS
self.chanceOfPickup = 3 #i.e. 1/5 or 0.2 probability
self.pickups = []

self.pickupHealth50Pixels = [[0, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0], [0, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0], [1, 0, 0, 0, 0, 0, 1,
0, 0, 0, 0, 0, 1], [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1], [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1], [0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0], [0, 0,
1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 0, 0, 1, 0, 1, 0, 0,
0, 0, 0], [0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0]]
self.pickupHealth100Pixels = [[0, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0], [0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0], [1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1], [0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0], [0,
0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0], [0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0], [0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 1, 1, 1, 0,
0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0]]
self.pickupScore50Pixels = [[0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 1], [0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1], [0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 1], [1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1], [0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1], [0, 0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 1]]
self.pickupScore100Pixels = [[0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1], [0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0], [0, 1, 0,
0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0], [1, 1, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0], [0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0], [0, 0, 0, 0, 0,
1, 0, 0, 1, 0, 1, 0, 1, 0], [0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1], [0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1]]
self.pickupScore500Pixels = [[0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1], [0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0], [0, 1, 0,
0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1], [1, 1, 1, 0, 0, 1, 1, 1, 0, 1, 0, 1, 0, 1], [0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0], [0, 0, 0, 0, 0,
0, 1, 0, 1, 0, 1, 0, 1, 0], [0, 0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 0, 1, 1]]

self.pickupTypes = [self.pickupHealth50Pixels, self.pickupHealth100Pixels, self.pickupScore50Pixels,
self.pickupScore100Pixels, self.pickupScore500Pixels, Gun1.pixels, Gun2.pixels, Gun3.pixels, Gun4.pixels]
self.bullets = []

#Weapon Switching
self.switchingWeapon = False

#Enemies
self.chanceOfEnemy = 3
self.maxEnemies = 20 #Upper limit from performance
self.enemies = []
```

```

self.enemyPixels0 = [[0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0], [0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0], [0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0], [0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0], [0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0], [1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1], [1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1], [1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0], [1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0], [1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0], [1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0], [0, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0]]

self.enemyPixels1 = [[0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1], [0, 1, 1, 0, 0, 0, 1, 1, 1, 1, 1, 1], [0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0], [1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0], [1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0], [0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0], [0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0], [0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0]]

self.enemyPixels2 = [[0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0], [0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0], [0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0], [0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0], [1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0], [1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1], [1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1], [0, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0], [0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0], [0, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 1, 1, 0, 0]]

self.enemyPixels3 = [[0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0], [0, 1, 1, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1], [1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1], [1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0], [1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0], [1, 0, 1, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0], [0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0], [0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0], [0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0], [0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0], [0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0]]

self.enemyPixels4 = [[0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1], [0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0], [0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0], [1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0], [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0], [0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0], [0, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0], [0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0], [0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0], [0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0], [0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0], [0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0]]

self.enemyTypes = [self.enemyPixels0, self.enemyPixels1, self.enemyPixels2, self.enemyPixels3, self.enemyPixels4]

#Enemies have built in weapon/turret so no pixels needed or both holder and barrel pos
EnemyGun0 = self.__class__.Gun(5, 100, 0.1, [], 0, 0, 12, 6, 5)
EnemyGun1 = self.__class__.Gun(4.2, 95, 0.4, [], 0, 0, 12, 0, 10)
EnemyGun2 = self.__class__.Gun(3.8, 85, 0.3, [], 0, 0, 14, 0, 10)
EnemyGun3 = self.__class__.Gun(3.1, 105, 0.4, [], 0, 0, 20, 4, 15)
EnemyGun4 = self.__class__.Gun(2.2, 100, 0.5, [], 0, 0, 25, 3, 35)

self.enemyGuns = [EnemyGun0, EnemyGun1, EnemyGun2, EnemyGun3, EnemyGun4]
self.enemyHealts = [50, 60, 75, 130, 200]

#Initial properties of one common enemy
#enemyPixels = self.enemyTypes[enemy.enemyTypeNum]
#enemyColliderWidth = len(self.enemyTypes[enemy.enemyTypeNum][0])
#enemyColliderHeight = len(self.enemyTypes[enemy.enemyTypeNum])

#Procedurally generate starting platforms
self.platforms = []
self.platformWidths = [35, 40, 45, 50, 55, 60, 65, 70, 80, 90, 95, 100, 110, 115] #Width of platform
self.platformHeight = 2
self.possiblePlatformHeights = [28, 30, 33, 35, 40, 43, 45, 48, 50] #Possible Yposes of platform
self.platformHeightDiff = 3 #The maximum change in height between platforms
self.distanceBetweenPlatforms = 25
self.platformsCovered = self.distanceBetweenPlatforms
self.reboundStrength = 2.2 # How far the player moves back when hitting the side of the platform
self.FirstPlatform = True
self.create_platforms(10) #generates 10 platforms in a block

#Camera

```

```

self.viewPortXpos = 0
self.followCameraDistance = 20
self.targetCameraDistance = 0

#General
self.playerRestart = False
self.restartCounter = 0
self.restartCounterLength = 40

#Score
self.SCORE = 0
self.A_SCORE = 0 #Keeps track of the score generated from killing enemies and pickups etc
#Health
self.HEALTH = self.playerProperties["maxHealth"]
self.healCounter = 0
self.healRate = 20

#Colour Flashes
self.flashCount = 0
self.flashLength = 0
self.flashColour = self.defaultColour

```

The draw function takes in the pixels, starting positions, the viewport (which is set to default to 0 for static images) and also a flipped variable that flips the shape along the left edge, i.e. when the player is turning to the left. For this method to work correctly it also assumes that pixels is in the specified binary array format, and will loop through the array, check whether the pixel is stored as a 1 (draws pixels) or 0 (skips to next pixel), offsetting each final pixel position by the initial position.

```

def draw(self, pixels, startXpos, startYpos, viewPortXpos=0, flippedY=False):
    #This function will draw the pixels starting from the top left corner
    if startXpos - viewPortXpos - len(pixels[0]) > self.WIDTH or startXpos - viewPortXpos + len(pixels[0]) < 0 or startYpos > self.HEIGHT: #Check within bounds
        return
    self.display.set_pen(15)
    multiplier = 1
    if flippedY:
        multiplier = -1

    for y in range(len(pixels)):
        for x in range(len(pixels[y])):
            if pixels[y][x] == 1:
                self.display.pixel(int(startXpos + (x*multiplier) - viewPortXpos), int(startYpos + y))

```

The main menu, help menu and game over screen use the sprites in the game to draw out a pre-determined layouts to make it fun and aesthetically pleasing, while including directions using the text function to navigate the user in the game. I also used different colours to indicate what panel the user is on and a small time delay so that the user doesn't accidentally jump back to the same panel. Furthermore, I created a standalone function clearing the display as I realised I needed to do frequently such as between frames and whenever switching between panels. *This segment of code works towards the **User Interface** objective:*

```

def main_menu(self):
    #PIXEL RAMPAGE
    self.clear_screen()
    self.gfx.set_backlight(*self.Green)
    self.display.set_pen(15)
    self.display.text("PIXEL RAMPAGE", 1, 1, scale=2.2)
    self.display.text("A - PLAY", 3, 20, scale=1)
    self.display.text("B - LEADERBOARD", 54, 20, scale=1)
    self.display.text("C - HELP", 48, 30, scale=1)

    self.draw(self.enemyPixels4, 1, 43)
    self.draw(self.enemyPixels1, 30, 43)

```

```

self.draw(self.enemyPixels1, 30, 54)
self.draw(self.enemyPixels0, 46, 50)
self.draw(self.playerFrame0, 59, 38)
self.draw(self.guns[4].pixels, 67, 42)
self.draw(self.enemyPixels0, 88, 50, 0, True)
self.draw(self.enemyPixels2, 105, 39, 0, True)
self.draw(self.enemyPixels2, 105, 52, 0, True)
self.draw(self.enemyPixels3, 127, 47, 0, True)
self.display.update()
while True:
    if self.gfx.switch_pressed(SWITCH_A):
        time.sleep(0.5)
        self.run()
    elif self.gfx.switch_pressed(SWITCH_B):
        time.sleep(0.5)
        self.show_leaderboard()
    elif self.gfx.switch_pressed(SWITCH_C):
        time.sleep(0.5)
        self.help_menu()

def help_menu(self):
    self.clear_screen()
    self.gfx.set_backlight(*self.Orange)
    self.display.set_pen(15)
    self.display.text("PIXEL RAMPAGE", 1, 1, scale=2.2)
    self.display.text("A - LEFT", 1, 17, scale=1)
    self.display.text("B - RIGHT", 1, 27, scale=1)
    self.display.text("C - SWITCH WEAPON->RESTART", 1, 37, scale=1)
    self.display.text("D - SHOOT", 1, 47, scale=1)
    self.display.text("E - JUMP->DOUBLE JUMP", 1, 57, scale=1)
    self.display.update()
    while True:
        if self.gfx.switch_pressed(SWITCH_A) or self.gfx.switch_pressed(SWITCH_B) or
self.gfx.switch_pressed(SWITCH_C) or self.gfx.switch_pressed(SWITCH_D) or self.gfx.switch_pressed(SWITCH_E):
            time.sleep(0.5)
            self.main_menu()

def game_over_screen(self):
    self.clear_screen()
    self.gfx.set_backlight(*self.Green)

    #Update leaderboard if needed
    scoreText = "{:06}".format(min(999999, self.SCORE))
    onLeaderboard = self.update_leaderboard(scoreText)

    self.display.set_pen(15)
    self.display.text("GAME OVER", 20, 1, scale=2.2)
    self.display.text("A - RESTART", 3, 20, scale=1)
    self.display.text("B - MAIN MENU", 65, 20, scale=1)
    if onLeaderboard:
        self.display.text("your on the leaderboard!!!!", 5, 31, scale=1)
    else:
        self.display.text("SCORE: ", 52, 31, scale=1)
    self.display.text(scoreText, 38, 40, scale=2)

    self.draw(self.enemyTypes[4], 1, 43)
    self.draw(self.guns[0].pixels, 33, 57)
    self.draw(self.guns[1].pixels, 48, 57)
    self.draw(self.guns[2].pixels, 63, 57)
    self.draw(self.guns[3].pixels, 78, 57)
    self.draw(self.guns[4].pixels, 93, 57)
    self.draw(self.enemyTypes[3], 127, 47, 0, True)
    self.display.update()
    while True:

```

```

if self.gfx.switch_pressed(SWITCH_A):
    #Restart
    time.sleep(0.5)
    self.__init__()
    self.run()
elif self.gfx.switch_pressed(SWITCH_B):
    #Main Menu
    time.sleep(0.5)
    self.__init__()
    self.main_menu()

def clear_screen(self):
    self.display.set_pen(0)
    self.display.clear()

```

I divided the leaderboard functionality into 2 subroutines including show_leaderboard where it will read the data and then show the top 10 scores onto the display. And an update_leaderboard function that will update the data if necessary and return True if the player's score is on the leaderboard. The format stores 10, 6 digit strings on separate lines to ensure data consistency and no data redundancy. I also had to use "r" and "w" file opening methods separately due to the raspberry pi pico does not support all methods. *This segment of code works towards the **leaderboard** objective:*

```

def show_leaderboard(self):
    self.clear_screen()
    self.gfx.set_backlight(*self.Red)
    self.display.set_pen(15)

    with open("leaderboard.txt", "r") as file:
        lines = file.readlines()
        lines = [line.strip() for line in lines]

    #Create leaderboard if empty/i.e. new device
    if lines == []:
        with open("leaderboard.txt", "w") as file:
            file.write("000000\n" * 10)
        lines = ["000000", "000000", "000000", "000000", "000000", "000000", "000000", "000000", "000000", "000000"]

    self.display.text("LEADERBOARD", 1, 1, scale=2.2)
    self.display.text("1. " + lines[0], 1, 17, scale=1)
    self.display.text("2. " + lines[1], 1, 27, scale=1)
    self.display.text("3. " + lines[2], 1, 37, scale=1)
    self.display.text("4. " + lines[3], 1, 47, scale=1)
    self.display.text("5. " + lines[4], 1, 57, scale=1)
    self.display.text("6. " + lines[5], 51, 17, scale=1)
    self.display.text("7. " + lines[6], 51, 27, scale=1)
    self.display.text("8. " + lines[7], 51, 37, scale=1)
    self.display.text("9. " + lines[8], 51, 47, scale=1)
    self.display.text("10. " + lines[9], 51, 57, scale=1)
    self.display.update()

    while True:
        if self.gfx.switch_pressed(SWITCH_A) or self.gfx.switch_pressed(SWITCH_B) or
self.gfx.switch_pressed(SWITCH_C) or self.gfx.switch_pressed(SWITCH_D) or self.gfx.switch_pressed(SWITCH_E):
            time.sleep(0.5)
            self.main_menu()

def update_leaderboard(self, newScore):
    with open("leaderboard.txt", "r") as file:
        lines = file.readlines()
        lines = [line.strip() for line in lines]

    #Create leaderboard if empty/i.e. new device
    if lines == []:
        with open("leaderboard.txt", "w") as file:
            file.write("000000\n" * 10)
        lines = ["000000", "000000", "000000", "000000", "000000", "000000", "000000", "000000", "000000", "000000"]

    if int(newScore) > int(lines[-1]):
        lines.pop()
        lines.append(newScore)
        lines.sort(key=int, reverse=True) #Sort list
        lines = list(map("{:06}".format, lines)) #Convert back to required format
        with open("leaderboard.txt", "w") as file:
            file.write("\n".join(lines)) #Store new data
        return True
    else:
        return False

```

This function spawns a given number of new platforms procedurally at random positions in the game. And then also randomly spawns enemies and pickups on those platforms. It also keeps a track of platforms covered so that new platforms can be generated at the optimal time, enhancing performance. No platforms are ever deleted, so that the player can re-trace progress. The chanceOfEnemies slowly increases overtime to make the game harder and the firstPlatform variable stops an enemy or pickup being spawned on the first platform. *This segment of code works towards the **Procedural generation of levels** objective:*

```

def create_platforms(self, numberOfPlatforms):
    for x in range(numberOfPlatforms):
        if len(self.platforms) == 0:
            prevXpos = 0
            prevYIndex = len(self.possiblePlatformHeights) // 2
        else:
            prevXpos = self.platforms[-1].Xpos + self.platforms[-1].width
            prevYIndex = self.possiblePlatformHeights.index(self.platforms[-1].Ypos)

        platform = self.__class__.Platform(prevXpos + self.distanceBetweenPlatforms,
            self.possiblePlatformHeights[max(0, min(random.randint(prevYIndex - self.platformHeightDiff,
                prevYIndex + self.platformHeightDiff), len(self.possiblePlatformHeights) - 1))],
            self.platformWidths[random.randrange(0, len(self.platformWidths))])
        self.platformsCovered += self.distanceBetweenPlatforms + platform.width
        self.platforms.append(platform)

    #Add pickups to platform
    isPickup = random.randint(1, self.chanceOfPickup)
    if isPickup == 1 and self.FirstPlatform == False:
        pickupTypeNum = random.randint(0, len(self.pickupTypes) - 1)
        pixels = self.pickupTypes[pickupTypeNum]
        pickup = self.__class__.Pickup(platform.Xpos + random.randint(0, platform.width - len(pixels[0])), platform.Ypos -
            len(pixels) - 1, pickupTypeNum)
        self.pickups.append(pickup)

    #SpawnEnemy
    isSpawnEnemy = random.randint(1, int(self.chanceOfEnemy))
    if isSpawnEnemy == 1 and len(self.enemies) < self.maxEnemies and self.FirstPlatform == False:
        enemyTypeNum = random.randint(0, len(self.enemyGuns) - 1)
        enemy = self.__class__.Enemy(enemyTypeNum, platform.Xpos + (platform.width // 2) -
            (len(self.enemyTypes[enemyTypeNum][0]) // 2), platform.Ypos - (len(self.enemyTypes[enemyTypeNum])), platform, 0,
            self.enemyGuns[enemyTypeNum].maxTimeBetweenShots, self.enemyHealts[enemyTypeNum])
        self.enemies.append(enemy)

    self.FirstPlatform = False
    self.chanceOfEnemy = max(1, self.chanceOfEnemy - 0.4)

```

The handle_input function is at the root of the game where different game variables are adjusted whenever a switch is pressed. I implemented a double jump mechanic as an additional feature as it thought it added more control to the player and allows them to jump further over platforms. This works based on waiting for a second button press while the player has already jumped of a platform and is in the air and then resetting all variables again when the player is grounded. *This segment of code works towards the **Movement** objective:*

```

def handle_input(self):
    #PLAYER MOVEMENT
    if self.playerProperties["canMoveX"]: #If player is falling of platform, player should no longer be able to move left or right
        if self.gfx.switch_pressed(SWITCH_A):
            self.playerProperties["Xspeed"] = min(max(-self.playerProperties["maxSpeed"], self.playerProperties["Xspeed"] -
                self.playerProperties["accel"]), -self.playerProperties["minSpeed"])
            elif self.gfx.switch_pressed(SWITCH_B):
                self.playerProperties["Xspeed"] = max(min(self.playerProperties["maxSpeed"], self.playerProperties["Xspeed"] +
                    self.playerProperties["accel"]), self.playerProperties["minSpeed"])
            else:
                if abs(self.playerProperties["Xspeed"]) <= 0.05: #Clamps Xspeed at small values
                    self.playerProperties["Xspeed"] = 0
                else:
                    self.playerProperties["Xspeed"] += -self.playerProperties["brakingForce"] if self.playerProperties["Xspeed"] >
                        0 else self.playerProperties["brakingForce"]

        if self.gfx.switch_pressed(SWITCH_E) and self.playerProperties["isJumping"] == False and
            self.playerProperties["hasJumped"] == False:
                #JUMP
                self.playerProperties["Yspeed"] = -self.playerProperties["jumpForce"]

```

```

    self.playerProperties["hasJumped"] = True

    if self.gfx.switch_pressed(SWITCH_E) == False and self.playerProperties["isJumping"] == True and
self.playerProperties["hasJumped"] == True:
        #Switch pressed again after jumping of a platform
        self.playerProperties["hasJumped"] = False
        self.playerProperties["readyToDoubleJump"] = True

    if self.gfx.switch_pressed(SWITCH_E) and self.playerProperties["isDoubleJumping"] == False and
self.playerProperties["readyToDoubleJump"] == True:
        #DOUBLE JUMPING
        self.playerProperties["Yspeed"] = -self.playerProperties["jumpForce"] *
self.playerProperties["doubleJumpMultiplier"]
        self.playerProperties["isDoubleJumping"] = True

    #SHOOTING
    self.gunProperties["isShooting"] = self.gfx.switch_pressed(SWITCH_D)

    #CHANGING WEAPON
    if self.gfx.switch_pressed(SWITCH_C) and self.gunProperties["isShooting"] == False and self.switchingWeapon ==
False:
        self.switchingWeapon = True

    #Loops through the currentWeapon list and cycles to the next one stored in currentWeaponNum
    if self.switchingWeapon == True and self.gfx.switch_pressed(SWITCH_C) == False:
        if self.gunProperties["currentWeaponNum"] != -1:
            currentWeaponIndex =
self.gunProperties["currentWeaponList"].index(self.gunProperties["currentWeaponNum"])
        else:
            currentWeaponIndex = -1
        if currentWeaponIndex + 1 < len(self.gunProperties["currentWeaponList"]):
            self.gunProperties["currentWeaponNum"] = self.gunProperties["currentWeaponList"][currentWeaponIndex + 1]
        else:
            self.gunProperties["currentWeaponNum"] = -1

        self.switchingWeapon = False

    #RESTARTING when holding SWITCH_C
    if self.gfx.switch_pressed(SWITCH_C):
        if self.restartCounter < self.restartCounterLength:
            self.restartCounter += 1
        else:
            self.playerRestart = True
    else:
        self.restartCounter = 0

```

The `closest_enemy` returns the enemy with the shortest distance away from the player in the direction that it is facing, and `None` if there are no enemies in range. This will be then be used to direct the player's bullets to the enemy and giving them a slight advantage therefore accounting for the lack of buttons. It does this by keeping track of a `closestDistance` variable and then comparing each next distance against this one, updating it an `closestEnemy` where neccessary.

```

def closest_enemy(self):
    closestDistance = math.inf
    closestEnemy = None

    #uses min and max to only identify enemies in the direction facing
    min = -math.inf
    max = math.inf

    if self.playerProperties["isDirectionRight"]:
        min = self.playerProperties["Xpos"]
    else:
        max = self.playerProperties["Xpos"]

    for enemy in self.enemies:
        if min <= enemy.enemyXpos <= max:
            distance = abs(enemy.enemyXpos - self.playerProperties["Xpos"])
            if distance < closestDistance:
                closestDistance = distance
                closestEnemy = enemy

    #100 is gun target lock max detection distance
    if closestDistance >= 100 or closestEnemy == None:
        return -1
    else:
        return closestEnemy

```

There are 2 seperate shooting functions, but they both have similar logic. shoot is called every frame however enemy_shoot is called when it is time for the enemy to shoot. They both work by instantiating Bullet objects at the barrel of the player/enemy and then adding it to the bullets list to be moved/updated in series with the other bullets, however the shoot function does have additional logic to calculate the angle down from the horizontal to the enemy-player path.

```

def shoot(self):
    if self.gunProperties["isShooting"] == True and self.gunProperties["currentWeaponNum"] != -1 and self.gunProperties["timeSinceLastShot"] <= 0:
        currentWeapon = self.guns[self.gunProperties["currentWeaponNum"]]
        closestEnemy = self.closest_enemy()
        if closestEnemy == -1: #If no enemy lock then shoots laterally
            angle = 0
        else:
            if self.playerProperties["isDirectionRight"]:
                startingPoint = self.playerProperties["Xpos"] + currentWeapon.gunHolderLocalXpos + currentWeapon.barrelLocalXpos
            else:
                startingPoint = self.playerProperties["Xpos"] - currentWeapon.gunHolderLocalXpos - currentWeapon.barrelLocalXpos - 1
            dx = closestEnemy.enemyXpos + (len(self.enemyTypes[closestEnemy.enemyTypeNum])[0]) // 2 + 5 - startingPoint
            dy = closestEnemy.enemyYpos - (len(self.enemyTypes[closestEnemy.enemyTypeNum]) // 2) - self.playerProperties["Ypos"] + currentWeapon.barrelLocalYpos

            #Calculates angle
            angle = (math.atan2(dy, dx))
            if not self.playerProperties["isDirectionRight"]:
                angle = angle - math.pi
            #Spawns bullets
            if self.playerProperties["isDirectionRight"] == True:
                bullet = self.__class__.Bullet(self.playerProperties["Xpos"] + currentWeapon.gunHolderLocalXpos + currentWeapon.barrelLocalXpos, self.playerProperties["Ypos"] + currentWeapon.barrelLocalYpos, currentWeapon.shootingSpeed, currentWeapon.range, True, currentWeapon.damage, angle)
            else:
                bullet = self.__class__.Bullet(self.playerProperties["Xpos"] - currentWeapon.gunHolderLocalXpos - currentWeapon.barrelLocalXpos - 1, self.playerProperties["Ypos"] + currentWeapon.barrelLocalYpos, currentWeapon.shootingSpeed, currentWeapon.range, False, currentWeapon.damage, angle)
            self.bullets.append(bullet)
            self.gunProperties["timeSinceLastShot"] = currentWeapon.maxTimeBetweenShots

def enemy_shoot(self, enemyXpos, enemyYpos, enemyTypeNum, enemyDirectionRight):

    currentWeapon = self.enemyGuns[enemyTypeNum]
    if enemyDirectionRight == True:
        bullet = self.__class__.Bullet(enemyXpos + currentWeapon.gunHolderLocalXpos + currentWeapon.barrelLocalXpos, enemyYpos + currentWeapon.barrelLocalYpos, currentWeapon.shootingSpeed, currentWeapon.range, enemyDirectionRight, currentWeapon.damage, 0)
    else:
        bullet = self.__class__.Bullet(enemyXpos - currentWeapon.gunHolderLocalXpos, enemyYpos + currentWeapon.barrelLocalYpos, currentWeapon.shootingSpeed, currentWeapon.range, enemyDirectionRight, currentWeapon.damage, 0)

    self.bullets.append(bullet)

```

Update objects is a general function that is run for bullets, pickups and enemies to loop through all the objects, update them or delete them if they have been marked as destroyed. It handles each object in the scene through looping through these object lists and dealing with each one individually. It is this approach that allowed me to not use asyncio or any other form of asynchronous programming. Here, it also checks whether the player has successfully killed an enemy, and then flashes the screen green as a confirmation message.

```

def update_objects(self, objects, objectNum):
    #ObjectNum: 1 - Bullets, 2 - Pickups, 3 - Enemies
    indexesToBeDestroyed = []

    for index in range(len(objects)):
        if objects[index].destroyed == True:
            indexesToBeDestroyed.append(index)
        else:
            if objectNum == 1:
                objects[index].update()
            elif objectNum == 3:
                objects[index].update(self.playerProperties["Xpos"], self)

    for index in sorted(indexesToBeDestroyed, reverse=True): #pops larger indexes first so that the smaller indexes don't
get effected
        if index < 0 or index > len(objects) - 1: continue
        object = objects.pop(index)
        if objectNum == 3: #Flash green to indicate that player killed enemy
            self.A_SCORE += ((object.enemyTypeNum + 1) * 100)
            self.flashColour = self.Green
            self.flashLength = 15
        del object

```

This function handles specifically the bullet objects by looping through them and comparing a bullet's position against the player, enemy and platform positions and subsequently destroying them or taking damage on the other objects if necessary. This also means that enemy's can kill other enemies as there is no distinction between enemy bullets and player bullets, adding a further layer of strategy to the game.

```

def damage_bullets(self):
    for bullet in self.bullets:
        #Check if hit player
        if self.is_overlapping(bullet.Xpos, bullet.Ypos, 2, 2, self.playerProperties["Xpos"], self.playerProperties["Ypos"],
            self.playerProperties["colliderWidth"], self.playerProperties["colliderHeight"]):
            self.flashColour = self.Red
            self.flashLength = 10
            self.HEALTH -= bullet.damage
            bullet.destroyed = True
            continue

        #Check if hit enemy
        for enemy in self.enemies:
            if self.is_overlapping(bullet.Xpos, bullet.Ypos, 2, 2, enemy.enemyXpos, enemy.enemyYpos,
                len(self.enemyTypes[enemy.enemyTypeNum][0]), len(self.enemyTypes[enemy.enemyTypeNum])):
                enemy.enemyHealth -= bullet.damage
                bullet.destroyed = True
                continue

        #Check if hit platform
        for platform in self.platforms:
            if self.is_overlapping(bullet.Xpos, bullet.Ypos, 2, 2, platform.Xpos, platform.Ypos, platform.width,
                self.platformHeight):
                bullet.destroyed = True
                continue

```

check_for_pickups calls the pickup function and destroys the associated Pickup object when the player comes into contact with one. Then the pickup function updates the score or health based upon the specific type of pick up that has been collected by using a switch like case by case statement. *This segment of code works towards the **Pickups** objective:*

```

def check_for_pickups(self):
    for pickup in self.pickups:
        #Check for any overlap between the two objects
        if self.is_overlapping(self.playerProperties["Xpos"], self.playerProperties["Ypos"], self.playerProperties["colliderWidth"],
self.playerProperties["colliderHeight"], pickup.Xpos, pickup.Ypos, len(self.pickupTypes[pickup.pickupTypeNum][0]),
len(self.pickupTypes[pickup.pickupTypeNum])) == True:
            pickup.destroyed = True
            self.pickup(pickup.pickupTypeNum)

def pickup(self, pickupTypeNum):
    if pickupTypeNum == 0:
        self.HEALTH = min(100, self.HEALTH + 50)
    elif pickupTypeNum == 1:
        self.HEALTH = 100
    elif pickupTypeNum == 2:
        self.A_SCORE += 50
    elif pickupTypeNum == 3:
        self.A_SCORE += 100
    elif pickupTypeNum == 4:
        self.A_SCORE += 500
    elif pickupTypeNum == 5:
        if 1 in self.gunProperties["currentWeaponList"]: return
        self.gunProperties["currentWeaponList"].append(1)
        self.gunProperties["currentWeaponNum"] = 1
    elif pickupTypeNum == 6:
        if 2 in self.gunProperties["currentWeaponList"]: return
        self.gunProperties["currentWeaponList"].append(2)
        self.gunProperties["currentWeaponNum"] = 2
    elif pickupTypeNum == 7:
        if 3 in self.gunProperties["currentWeaponList"]: return
        self.gunProperties["currentWeaponList"].append(3)
        self.gunProperties["currentWeaponNum"] = 3
    elif pickupTypeNum == 8:
        if 4 in self.gunProperties["currentWeaponList"]: return
        self.gunProperties["currentWeaponList"].append(4)
        self.gunProperties["currentWeaponNum"] = 4

```

update_physics will increase the player's Y speed downwards when the player is in the air. update_player, is used to prevent the player from falling through or hitting the sides of the platform. To do this I loop over all platforms and see if there is any overlap the 2 objects. Then it identifies if the player is hitting the platform from the side or top. If the player hits, the side, there is additional logic to freeze the player and push them back, giving the effect that they are unconscious and falling based upon the rebound strength. I also wrote a pre-defined overlap function that takes in the positions and widths and heights of 2 objects and return true when they overlap.

```

def update_physics(self):
    if self.playerProperties["isJumping"]:
        self.playerProperties["Yspeed"] += self.physicalProperties["Gravity"] #Applying Gravity

def update_player(self):
    #New Collision detection code
    #We need to check the bottom of the player with the height of the platform
    isGrounded = False

    for platform in self.platforms:
        if platform.Xpos - self.viewPortXpos > self.WIDTH or platform.Xpos - self.viewPortXpos + platform.width < 0:
            continue
        if self.is_overlapping(self.playerProperties["Xpos"], self.playerProperties["Ypos"], self.playerProperties["colliderWidth"],
self.playerProperties["colliderHeight"],
platform.Xpos, platform.Ypos, platform.width, self.platformHeight):

            if (self.playerProperties["Ypos"] <= platform.Ypos and
self.playerProperties["Ypos"] + self.playerProperties["colliderHeight"] >= platform.Ypos + self.platformHeight):
                if self.playerProperties["Xspeed"] > 0:
                    self.playerProperties["Xpos"] -= self.reboundStrength * abs(self.playerProperties["Xspeed"])
                elif self.playerProperties["Xspeed"] < 0:
                    self.playerProperties["Xpos"] += self.reboundStrength * abs(self.playerProperties["Xspeed"])
                self.playerProperties["canMoveX"] = False

            else:
                # Top collision (landing)
                if self.playerProperties["Yspeed"] > 0:
                    isGrounded = True

    if isGrounded == False or self.playerProperties["canMoveX"] == False:
        self.playerProperties["isJumping"] = True
    else:
        self.playerProperties["Yspeed"] = 0
        self.playerProperties["isJumping"] = False
        self.playerProperties["isDoubleJumping"] = False
        self.playerProperties["readyToDoubleJump"] = False

    if self.playerProperties["canMoveX"]:
        self.playerProperties["Xpos"] += self.playerProperties["Xspeed"]

    self.playerProperties["Ypos"] += self.playerProperties["Yspeed"]

    #UpdateDirection
    if self.playerProperties["Xspeed"] > 0:
        self.playerProperties["isDirectionRight"] = True
    elif self.playerProperties["Xspeed"] < 0:
        self.playerProperties["isDirectionRight"] = False

def is_overlapping(self, x1, y1, w1, h1, x2, y2, w2, h2):
    return not (
        x1 + w1 < x2 or # obj1 is completely to the left of obj2
        x1 > x2 + w2 or # obj1 is completely to the right of obj2
        y1 + h1 < y2 or # obj1 is completely above obj2
        y1 > y2 + h2 # obj1 is completely below obj2
    )

```

The update_camera method uses the targetCameraDistance to keep track of where the camera should be. The camera should be in behind the player when moving to the left and in front of the player when moving to the right. It will move further away from the player at higher speeds, to further enhance the effect that the player is accelerating. Then it linearly interpolates the actual followCameraDistance to the target one to make transitions smooth.

change_player_frame will update the next frame to be displayed if SWITCH_A or SWITCH_B is pressed. The

next frame is wrapped around using modular arithmetic, so that after reaching the last frame it loops back to the first and repeats.

```
def update_camera(self):
    if self.playerProperties["isDirectionRight"]:
        self.targetCameraDistance = max(25, 20 + self.playerProperties["Xspeed"] * 14.2)
    else:
        self.targetCameraDistance = min(self.WIDTH - 25, self.WIDTH - (20 + self.playerProperties["Xspeed"] * 14.2))

    # Smoothly move the followCameraDistance to targetCameraDistance
    self.followCameraDistance += (self.targetCameraDistance - self.followCameraDistance) * 0.022

def update_map(self):
    if int(self.playerProperties["Xpos"]) + int(1.1 * self.WIDTH) > self.platformsCovered:
        self.create_platforms(10)

def change_player_frame(self):
    if (not self.gfx.switch_pressed(SWITCH_A) and not self.gfx.switch_pressed(SWITCH_B)) or not self.playerProperties["canMoveX"]:
        self.frameNum = 0
        return
    if self.frameCounter < self.frameLength:
        self.frameCounter += 1
    else:
        self.frameNum = (self.frameNum + 1) % len(self.playerFrames)
        self.frameCounter = 0
```

In `render_game`, all objects are drawn out based on their current positions and the camera's current position. This function also changes the `frameLength` of the animation so that the player's legs move faster as the speed increases, however this is hard to tell when running the game. Specifically, when drawing guns and enemies, it needs to change the start position depending if the player/object is facing left or right adding slight complexities. It also call `update_UI` at the end to make sure UI is rendered over the game.

```

def render_game(self):
    self.viewPortXpos = self.playerProperties["Xpos"] - self.followCameraDistance

    if self.playerProperties["Xspeed"] == 0:
        self.frameLength = self.maxFrameLength
    else:
        self.frameLength = min(self.maxFrameLength, abs(int(self.maxFrameLength / self.playerProperties["Xspeed"])))

    #Clear Display
    self.clear_screen()

    #Set pen to black
    self.display.set_pen(15)
    #Draw Enviroment
    #Draw Platforms
    for platform in self.platforms:
        if platform.Xpos - self.viewPortXpos > self.WIDTH or platform.Xpos - self.viewPortXpos + platform.width < 0:
            continue
        self.display.rectangle(int(platform.Xpos - self.viewPortXpos), platform.Ypos, platform.width, self.platformHeight)

    #Draw Player
    if self.playerProperties["isDirectionRight"]:
        self.draw(self.playerFrames[self.frameNum], int(self.playerProperties["Xpos"]), int(self.playerProperties["Ypos"]),
self.viewPortXpos)
    else:
        self.draw(self.playerFrames[self.frameNum], int(self.playerProperties["Xpos"] + self.playerProperties["colliderWidth"] -
1), int(self.playerProperties["Ypos"]), self.viewPortXpos, True)
    #Draw Gun
    if self.gunProperties["currentWeaponNum"] != -1:
        currentWeapon = self.guns[self.gunProperties["currentWeaponNum"]]
        if self.playerProperties["isDirectionRight"] == True:
            self.draw(currentWeapon.pixels, int(self.playerProperties["Xpos"]) + self.playerProperties["colliderWidth"] +
currentWeapon.gunHolderLocalXpos, self.playerProperties["Ypos"] + currentWeapon.gunHolderLocalYpos, self.viewPortXpos)
        else:
            self.draw(currentWeapon.pixels, int(self.playerProperties["Xpos"]) - currentWeapon.gunHolderLocalXpos - 1,
self.playerProperties["Ypos"] + currentWeapon.gunHolderLocalYpos, self.viewPortXpos, True)

    #Draw Bullets
    self.display.set_pen(15)
    for bullet in self.bullets:
        self.display.rectangle(int(bullet.Xpos - self.viewPortXpos), int(bullet.Ypos), 2, 2)

    #Draw pickups
    for pickup in self.pickups:
        self.draw(self.pickupTypes[pickup.pickupTypeNum], pickup.Xpos, pickup.Ypos, self.viewPortXpos)

    #Draw enemies
    for enemy in self.enemies:
        if enemy.enemyDirectionRight == True:
            self.draw(self.enemyTypes[enemy.enemyTypeNum], enemy.enemyXpos, enemy.enemyYpos, self.viewPortXpos)
        else:
            self.draw(self.enemyTypes[enemy.enemyTypeNum], enemy.enemyXpos +
len(self.enemyTypes[enemy.enemyTypeNum][0]), enemy.enemyYpos, self.viewPortXpos, True)

    self.update_UI()

    self.display.update() # Update display with the above items

```

update_default_colour changes the backlight colour of the screen and works in conjunction with the health bar to indicate the users current health and flash_colour can be called to temporarily change the colour, i.e. when the player takes damage. I positioned the score counter in the left hand corner so that larger scores wouldn't be cut off by the edge of the display. update_UI draws out a health bar which works by drawing a percentage of a larger rectangle based upon the player's current health.

```

def update_default_colour(self):
    if self.HEALTH >= 75:
        self.defaultColour = self.Blue
    elif self.HEALTH >= 45:
        self.defaultColour = self.Orange
    else:
        self.defaultColour = self.Red

def flash_colour(self):
    if self.flashColour == self.defaultColour:
        self.gfx.set_backlight(*self.flashColour)
        return

    if self.flashCount > self.flashLength:
        self.flashCount = 0
        self.gfx.set_backlight(*self.defaultColour)
        self.flashColour = self.defaultColour
    else:
        self.gfx.set_backlight(*self.flashColour)
        self.flashCount += 1

def update_UI(self):
    self.display.set_pen(15)
    #SCORE
    self.SCORE = max(self.SCORE - self.A_SCORE, int(self.playerProperties["Xpos"] -
self.playerProperties["originalXpos"])) + self.A_SCORE

    self.flash_colour() #runs every second
    text = "SCORE: " + str(self.SCORE)
    self.display.text(text, 1, 1, scale=0.8)

    #HEALTH BAR
    self.display.rectangle(self.WIDTH - 51, self.HEIGHT - 9, 50, 8)
    self.display.set_pen(0)
    self.display.rectangle(self.WIDTH - 50, self.HEIGHT - 8, 48, 6)
    self.display.set_pen(15)
    self.display.rectangle(self.WIDTH - 49, self.HEIGHT - 7, ((self.HEALTH) * 46) // 100, 4)

```

is_Dead returns True when the player restarts/falls of the platform/health depletes below 0. heal will increase the health value slowly and gradually over time, so that the player's health increases even if no health pickups are spawned. *This segment of code works towards the **Health** objective:*

```

def is_Dead(self):
    if self.playerRestart:
        return True
    if (self.playerProperties["Ypos"] >= self.HEIGHT or self.HEALTH <= 0):
        #RESTART GAME
        return True
    return False

def heal(self):
    if self.healCounter < self.healRate:
        self.healCounter += 1
    else:
        self.HEALTH = min(100, self.HEALTH + 1)
        self.healCounter = 0

```

Run program calls each method in order while the player is not dead. An issue I had was the game's frame rate dropping significantly when many large enemies where on the screen at the same time. Hence i used time.sleep to fix a constant frame rate of 60FPS (60 frames per second) through out the game.

```

def run(self):
    self.clear_screen()
    time.sleep(0.5)
    while not self.is_Dead():
        self.handle_input()
        self.update_physics()
        self.update_player()
        self.check_for_pickups()
        self.shoot()
        self.update_objects(self.bullets, 1)
        self.damage_bullets()
        self.update_objects(self.pickups, 2)
        self.update_objects(self.enemies, 3)
        self.update_camera()
        self.update_map()
        self.update_default_colour()
        self.change_player_frame()
        self.heal()
        #self.debug()
        self.render_game()
        frameRate = 1/60
        self.gunProperties["timeSinceLastShot"] -= frameRate
        #Game speed, i.e. waits for a fixes time before cycling (60FPS)
        time.sleep(frameRate)
    time.sleep(0.5)
    self.game_over_screen()

```

This part is what is called by the program when it is first run. It creates a game class and then opens the main menu. (The commented code here can be used to reset the leaderboard completely).

```

if __name__ == "__main__":
    #del leaderboard
    #with open("leaderboard.txt", "w") as file:
    # file.write("")

    game = Game()
    game.main_menu()

```

Section 4 - Testing

What is testing?

In order to make sure all of my specification points are completed, I need to test certain aspects of my program. In this section, I will create targeted tests in order to make my program more robust using test-driven design to continously improve my program, and keep track of progress.

Testing Table

Testing no.	Test Name	Objective being tested	Reason for test	Expected result	Actual result	Video Evidence
1	Basic Movement	Movement	To ensure I can move the character correctly in all directions	Player can use controls to move player	PARTIALLY SUCCESSFUL (doesn't fuffill the full objective which was eventually achieved in Complex Movement and Camera Follow)	Test1
2	Basic Shooting	Shooting	To ensure bullets are being spawned and updated as expected	On click bullets are spawning one after each other, and then moving to the right	PARTIALLY SUCCESSFUL (doesn't fuffill the full objective which was eventually achieved in Complex Shooting and Weapon Switching)	Test2
3	Map Creation	Procedural Generation of Levels	Platforms should be spawned in of different widths and at varied	Player's can move between platforms which are continously generated as the	SUCCESFUL	Test3

Testing no.	Test Name	Objective being tested	Reason for test	Expected result	Actual result	Video Evidence
4	Complex Movement and Camera Follow	Movement	Player should be able to <i>accelerate</i> and <i>double jump</i> building upon the existing movement	Player should be able to move smoothly left and right with the Camera keeping the player in view at all times. Additionally there should be collision detection between player and platform	SUCCESSFUL	Test4
5	Spawning of Pickups	Pickups	On platforms, pickups should be randomly spawned along the platform and should be collected by the player	Player collecting pickup and then is destroyed	SUCCESSFUL	Test5
6	Complex Shooting and Weapon Switching	Shooting	Player should be able to switch weapon and fire, with each weapon having different controllable properties	Different weapons that can be picked and then used	SUCCESSFUL	Test6
				Enemies can shoot bullets and move along		

Testing no.	Test Name	Objective being tested	Reason for test	Expected result	Actual result	Video Evidence
7	Spawning/Moving/Shooting of Enemies	Enemies	Enemies are used in this game to increase difficulty and oppose the player so their functionality is essential	the platform to follow and target the player. Additionally, they should be deleted if the player shoots enough bullets to kill them	SUCCESSFUL	Test7
8	Player's Score Increases Overtime	Scoring	The score should be consistent and update correctly	As player moves to the right/cover' s a larger distance the current score increases or increases when collecting certain pickups and even when killing enemies	SUCCESSFUL	Test8
9	Player Health Updates Correctly	Health	Health in the game is affected by multiple factors including bullet damage/pickups/healing over time/falling off platform	The player's health should react appropriately and according to each situation	SUCCESSFUL	Test9
10	Help Pop Up Panel	Ease of Use	This panel is needed for first-time users to understand how to play the game	Player's can enter and exit this panel with ease	SUCCESSFUL	Test10

Testing no.	Test Name	Objective being tested	Reason for test	Expected result	Actual result	Video Evidence
11	Player animations	Animation	Player should be animated to keep the game engaging and add a sense of movement	When player is moving, the animation is working and cycling through all frames	SUCCESSFUL	Test11
12	Navigation	User interface	The player should be able to access each part of the game easily	User navigates to every panel without any issues	SUCCESSFUL	Test12
13	Restarting the Game	Restart function	Player should be able to restart the game on command /submit score	Restarts game on click	SUCCESSFUL	Test13
14	Leaderboard is Displayed and Updated Correctly	Leaderboard	To test that leaderboard updates/reads and stores data effectively	User should be able to view current leaderboard and have their score updated on it if it is high enough. The device also stores leaderboard locally on a text file	SUCCESSFUL	Test14

Code for each Test

For each test in the table above, here is the associated code:

Test 1 - Basic Movement

```

from gfx_pack import GfxPack, SWITCH_A, SWITCH_B, SWITCH_C, SWITCH_D, SWITCH_E
import time

class Game:
    def __init__(self):
        #Initialize the hardware
        self.gfx = GfxPack()
        self.display = self.gfx.display
        self.WIDTH, self.HEIGHT = self.display.get_bounds()

        self.counter = 0
        self.gfx.set_backlight(0, 255, 0)

        #Set Font
        self.display.set_font("bitmap8")

    def handle_input(self):
        if self.gfx.switch_pressed(SWITCH_A):
            self.counter += 1

    def render_game(self):
        self.display.set_pen(0)
        self.display.clear()

        if self.counter % 2 == 0:
            self.display.set_pen(15)
            self.display.text(str(self.counter), 0, 0)
            self.display.update() # Update display with the above items

    def run(self):
        while True:
            self.handle_input()
            self.render_game()

            #Game speed, i.e. waits for 0.01s before cycling
            time.sleep(0.01)

if __name__ == "__main__":
    game = Game()
    game.run()

```

Test 2 - Shooting

```

def shoot(self):
    if self.gunProperties["isShooting"] == True and self.gunProperties["currentWeaponNum"] != -1 and
self.gunProperties["timeSinceLastShot"] <= 0:
        currentWeapon = self.guns[self.gunProperties["currentWeaponNum"]]
        bullet = self.__class__.Bullet(self.playerProperties["Xpos"] + currentWeapon.barrelLocalXpos,
self.playerProperties["Ypos"] + currentWeapon.barrelLocalYpos, currentWeapon.shootingSpeed, currentWeapon.range)
        self.bullets.append(bullet)
        self.gunProperties["timeSinceLastShot"] = currentWeapon.maxTimeBetweenShots

def update_bullets(self):
    bullets = self.bullets
    indexesToBeDestroyed = []

    for bulletIndex in range(len(bullets)):
        if bullets[bulletIndex].destroyed == True:
            indexesToBeDestroyed.append(bulletIndex)
        else:
            bullets[bulletIndex].update()

    for bulletIndex in indexesToBeDestroyed:
        bullet = bullets.pop(bulletIndex)
        del bullet

#Draw Gun
if self.gunProperties["currentWeaponNum"] != -1:
    currentWeapon = self.guns[self.gunProperties["currentWeaponNum"]]
    self.draw(currentWeapon.pixels, self.playerProperties["Xpos"] + currentWeapon.gunHolderLocalXpos,
self.playerProperties["Ypos"] + currentWeapon.gunHolderLocalYpos)

#Draw Bullets
self.display.set_pen(15)
for bullet in self.bullets:
    self.display.rectangle(int(bullet.Xpos), int(bullet.Ypos), 2, 2)

```

Test 3 - Map Creation

```

def create_platforms(self, numberOfPlatforms):
    for x in range(numberOfPlatforms):
        if len(self.platforms) == 0:
            prevXpos = 0
            prevYIndex = len(self.possiblePlatformHeights) // 2
        else:
            prevXpos = self.platforms[-1].Xpos + self.platforms[-1].width
            prevYIndex = self.possiblePlatformHeights.index(self.platforms[-1].Ypos)

        platform = self.__class__.Platform(prevXpos + self.distanceBetweenPlatforms,
self.possiblePlatformHeights[max(0, min(random.randint(prevYIndex - self.platformHeightDiff,
prevYIndex + self.platformHeightDiff), len(self.possiblePlatformHeights) - 1))],
self.platformWidths[random.randrange(0, len(self.platformWidths))])
        self.platformsCovered += self.distanceBetweenPlatforms + platform.width
        self.platforms.append(platform)

```

Test 4 - Complex Movement and Follow

```

self.playerProperties = {
    "Xpos" : 30,      #Spawn Position, Xpos and Ypos mark the top left hand corner of the player
    "originalXpos" : 30,
    "Ypos" : 9,
    "colliderWidth" : 9,
    "colliderHeight" : 16,
    #MOVEMENT
    "Xspeed" : 0,
    "Ysneed" : 0.

```

```

        "speed" : 0,
        "accel" : 0.038,
        "minSpeed" : 0.2,
        "maxSpeed" : 1.8,
        "brakingForce" : 0.012,      #The lower this value, the easier it is to slide
        "jumpForce" : 0.38,
        "isJumping" : True,
        "readyToDoubleJump" : False,
        "hasJumped" : False,
        "isDoubleJumping" : True,
        "doubleJumpMultiplier" : 1.5,
        "isDirectionRight" : True,
        "canMoveX" : True,
        "maxHealth" : 100
    }
}

```

...

```

def handle_input(self):
    #PLAYER MOVEMENT
    if self.playerProperties["canMoveX"]:
        if self.gfx.switch_pressed(SWITCH_A):
            self.playerProperties["Xspeed"] = min(max(-self.playerProperties["maxSpeed"], self.playerProperties["Xspeed"] -
self.playerProperties["accel"]), -self.playerProperties["minSpeed"])
        elif self.gfx.switch_pressed(SWITCH_B):
            self.playerProperties["Xspeed"] = max(min(self.playerProperties["maxSpeed"], self.playerProperties["Xspeed"] +
self.playerProperties["accel"]), self.playerProperties["minSpeed"])
        else:
            if abs(self.playerProperties["Xspeed"]) <= 0.05:
                self.playerProperties["Xspeed"] = 0
            else:
                self.playerProperties["Xspeed"] += -self.playerProperties["brakingForce"] if self.playerProperties["Xspeed"] > 0
    else self.playerProperties["brakingForce"]

    if self.gfx.switch_pressed(SWITCH_E) and self.playerProperties["isJumping"] == False and
self.playerProperties["hasJumped"] == False:

        self.playerProperties["Yspeed"] = -self.playerProperties["jumpForce"]
        self.playerProperties["hasJumped"] = True

    if self.gfx.switch_pressed(SWITCH_E) == False and self.playerProperties["isJumping"] == True and
self.playerProperties["hasJumped"] == True:
        self.playerProperties["hasJumped"] = False
        self.playerProperties["readyToDoubleJump"] = True

    if self.gfx.switch_pressed(SWITCH_E) and self.playerProperties["isDoubleJumping"] == False and
self.playerProperties["readyToDoubleJump"] == True:
        self.playerProperties["Yspeed"] = -self.playerProperties["jumpForce"] * self.playerProperties["doubleJumpMultiplier"]
        self.playerProperties["isDoubleJumping"] = True

```

...

```

def update_player(self):
    #New Collision detection code
    #We need to check the bottom of the player with the height of the platform
    isGrounded = False

    for platform in self.platforms:
        if platform.Xpos - self.viewPortXpos > self.WIDTH or platform.Xpos - self.viewPortXpos + platform.width < 0:
            continue
        if self.is_overlapping(self.playerProperties["Xpos"], self.playerProperties["Ypos"],
self.playerProperties["colliderWidth"], self.playerProperties["colliderHeight"],
platform.Xpos, platform.Ypos, platform.width, self.platformHeight):

            if (self.playerProperties["Ypos"] <= platform.Ypos and

```

```

self.playerProperties["Ypos"] + self.playerProperties["colliderHeight"] >= platform.Ypos + self.platformHeight):
if self.playerProperties["Xspeed"] > 0:
    self.playerProperties["Xpos"] -= self.reboundStrength * abs(self.playerProperties["Xspeed"])
elif self.playerProperties["Xspeed"] < 0:
    self.playerProperties["Xpos"] += self.reboundStrength * abs(self.playerProperties["Xspeed"])
self.playerProperties["canMoveX"] = False

else:
    # Top collision (landing)
    if self.playerProperties["Yspeed"] > 0:
        isGrounded = True

if isGrounded == False or self.playerProperties["canMoveX"] == False:
    self.playerProperties["isJumping"] = True
else:
    self.playerProperties["Yspeed"] = 0
    self.playerProperties["isJumping"] = False
    self.playerProperties["isDoubleJumping"] = False
    self.playerProperties["readyToDoubleJump"] = False

if self.playerProperties["canMoveX"]:
    self.playerProperties["Xpos"] += self.playerProperties["Xspeed"]

self.playerProperties["Ypos"] += self.playerProperties["Yspeed"]

#UpdateDirection
if self.playerProperties["Xspeed"] > 0:
    self.playerProperties["isDirectionRight"] = True
elif self.playerProperties["Xspeed"] < 0:
    self.playerProperties["isDirectionRight"] = False

```

Test 5 - Spawning of Pickups

```

#Add pickups to platform
isPickup = random.randint(1, self.chanceOfPickup)
if isPickup == 1 and self.FirstPlatform == False:
    pickupTypeNum = random.randint(0, len(self.pickupTypes) - 1)
    pixels = self.pickupTypes[pickupTypeNum]
    pickup = self.__class__.Pickup(platform.Xpos + random.randint(0, platform.width - len(pixels[0])), platform.Ypos -
len(pixels) - 1, pickupTypeNum)
    self.pickups.append(pickup)

def check_for_pickups(self):
    for pickup in self.pickups:
        #Check for any overlap between the two objects
        if self.is_overlapping(self.playerProperties["Xpos"], self.playerProperties["Ypos"], self.playerProperties["colliderWidth"],
self.playerProperties["colliderHeight"], pickup.Xpos, pickup.Ypos, len(self.pickupTypes[pickup.pickupTypeNum][0]),
len(self.pickupTypes[pickup.pickupTypeNum])) == True:
            pickup.destroyed = True
            self.pickedup(pickup.pickupTypeNum)

def pickup(self, pickupTypeNum):
    if pickupTypeNum == 0:
        self.HEALTH = min(100, self.HEALTH + 50)
    elif pickupTypeNum == 1:
        self.HEALTH = 100
    elif pickupTypeNum == 2:
        self.A_SCORE += 50
    elif pickupTypeNum == 3:
        self.A_SCORE += 100
    elif pickupTypeNum == 4:
        self.A_SCORE += 500
    elif pickupTypeNum == 5:
        if 1 in self.gunProperties["currentWeaponList"]: return
        self.gunProperties["currentWeaponList"].append(1)
        self.gunProperties["currentWeaponNum"] = 1
    elif pickupTypeNum == 6:
        if 2 in self.gunProperties["currentWeaponList"]: return
        self.gunProperties["currentWeaponList"].append(2)
        self.gunProperties["currentWeaponNum"] = 2
    elif pickupTypeNum == 7:
        if 3 in self.gunProperties["currentWeaponList"]: return
        self.gunProperties["currentWeaponList"].append(3)
        self.gunProperties["currentWeaponNum"] = 3
    elif pickupTypeNum == 8:
        if 4 in self.gunProperties["currentWeaponList"]: return
        self.gunProperties["currentWeaponList"].append(4)
        self.gunProperties["currentWeaponNum"] = 4

```

Test 6 - Complex Shooting Mechanics and Weapon Switching

```

self.gunProperties = {
    #non-specific
    "isShooting" : False,
    "currentWeaponNum" : 0,          #The current weapon is -1 when the character has no weapon, we will start with the most
basic weapon Gun0
    "timeSinceLastShot" : 0,
    "currentWeaponList" : [0]
}
...

#SHOOTING
self.gunProperties["isShooting"] = self.gfx.switch_pressed(SWITCH_D)

#CHANGING WEAPON
if self.gfx.switch_pressed(SWITCH_C) and self.gunProperties["isShooting"] == False and self.switchingWeapon == False:

```

```

if self.gfx.switch_pressed(SWITCH_C) and self.gunProperties["isShooting"] == False and self.switchingWeapon == False:
    self.switchingWeapon = True

if self.switchingWeapon == True and self.gfx.switch_pressed(SWITCH_C) == False:
    if self.gunProperties["currentWeaponNum"] != -1:
        currentWeaponIndex = self.gunProperties["currentWeaponList"].index(self.gunProperties["currentWeaponNum"])
    else:
        currentWeaponIndex = -1
    if currentWeaponIndex + 1 < len(self.gunProperties["currentWeaponList"]):
        self.gunProperties["currentWeaponNum"] = self.gunProperties["currentWeaponList"][currentWeaponIndex + 1]
    else:
        self.gunProperties["currentWeaponNum"] = -1

    self.switchingWeapon = False

...

def shoot(self):
    if self.gunProperties["isShooting"] == True and self.gunProperties["currentWeaponNum"] != -1 and
self.gunProperties["timeSinceLastShot"] <= 0:
        currentWeapon = self.guns[self.gunProperties["currentWeaponNum"]]
        closestEnemy = self.closest_enemy()
        if closestEnemy == -1:
            angle = 0
        else:
            if self.playerProperties["isDirectionRight"]:
                startingPoint = self.playerProperties["Xpos"] + currentWeapon.gunHolderLocalXpos +
currentWeapon.barrelLocalXpos
            else:
                startingPoint = self.playerProperties["Xpos"] - currentWeapon.gunHolderLocalXpos -
currentWeapon.barrelLocalXpos - 1
            dx = closestEnemy.enemyXpos + (len(self.enemyTypes[closestEnemy.enemyTypeNum][0]) // 2) + 5 - startingPoint
            dy = closestEnemy.enemyYpos - (len(self.enemyTypes[closestEnemy.enemyTypeNum]) // 2) -
self.playerProperties["Ypos"] + currentWeapon.barrelLocalYpos

            angle = (math.atan2(dy, dx))
            if not self.playerProperties["isDirectionRight"]:
                angle = angle - math.pi

            if self.playerProperties["isDirectionRight"] == True:
                bullet = self.__class__.Bullet(self.playerProperties["Xpos"] + currentWeapon.gunHolderLocalXpos +
currentWeapon.barrelLocalXpos, self.playerProperties["Ypos"] + currentWeapon.barrelLocalYpos,
currentWeapon.shootingSpeed, currentWeapon.range, True, currentWeapon.damage, angle)
            else:
                bullet = self.__class__.Bullet(self.playerProperties["Xpos"] - currentWeapon.gunHolderLocalXpos -
currentWeapon.barrelLocalXpos - 1, self.playerProperties["Ypos"] + currentWeapon.barrelLocalYpos,
currentWeapon.shootingSpeed, currentWeapon.range, False, currentWeapon.damage, angle)
            self.bullets.append(bullet)
            self.gunProperties["timeSinceLastShot"] = currentWeapon.maxTimeBetweenShots

...

def damage_bullets(self):
    for bullet in self.bullets:
        #Check if hit player
        if self.is_overlapping(bullet.Xpos, bullet.Ypos, 2, 2, self.playerProperties["Xpos"], self.playerProperties["Ypos"],
self.playerProperties["colliderWidth"], self.playerProperties["colliderHeight"]):
            self.flashColour = self.Red
            self.flashLength = 10
            self.HEALTH -= bullet.damage
            bullet.destroyed = True
            continue

        #Check if hit enemy

```

```

#Check if hit enemy
for enemy in self.enemies:
    if self.is_overlapping(bullet.Xpos, bullet.Ypos, 2, 2, enemy.enemyXpos, enemy.enemyYpos,
len(self.enemyTypes[enemy.enemyTypeNum][0]), len(self.enemyTypes[enemy.enemyTypeNum])):
        enemy.enemyHealth -= bullet.damage
        bullet.destroyed = True
        continue

#Check if hit platform
for platfrom in self.platforms:
    if self.is_overlapping(bullet.Xpos, bullet.Ypos, 2, 2, platfrom.Xpos, platfrom.Ypos, platfrom.width,
self.platformHeight):
        bullet.destroyed = True
        continue

```

Test 7 - Spawning/Movement/Shooting of Enemies

```

class Enemy:
    def __init__(self, enemyTypeNum, Xpos, Ypos, platform, weaponNum, enemyFireRate, enemyHealth):
        self.enemyTypeNum = enemyTypeNum
        self.enemyXpos = Xpos
        self.enemyYpos = Ypos
        self.detectionDistance = 100
        self.fireRandomness = 5
        self.platform = platform
        self.enemyfireRate = enemyFireRate
        self.enemyTimeSinceLastShot = 0
        self.enemyHealth = enemyHealth
        self.enemyDirectionRight = False
        self.closestDistanceToPlayer = 16
        self.destroyed = False

    def update(self, playerXpos, GameScene):
        self.enemyDirectionRight = True if playerXpos - self.enemyXpos >= 0 else False

        if self.enemyHealth <= 0:
            self.destroyed = True
            return

        #Fire
        if abs(self.enemyXpos - playerXpos) <= self.detectionDistance:
            if self.enemyTimeSinceLastShot > self.enemyfireRate:
                self.enemyTimeSinceLastShot = random.randint(-self.fireRandomness, self.fireRandomness) / 100
                GameScene.enemy_shoot(self.enemyXpos, self.enemyYpos, self.enemyTypeNum, self.enemyDirectionRight)
            else:
                self.enemyTimeSinceLastShot += 0.01

        #Move
        multiplier = 1 if self.enemyDirectionRight == True else -1
        self.targetPosition = max(self.platform.Xpos, min(playerXpos - multiplier *
(len(GameScene.enemyTypes[self.enemyTypeNum][0]) + self.closestDistanceToPlayer) , self.platform.Xpos +
self.platform.width - len(GameScene.enemyTypes[self.enemyTypeNum][0])))
        self.enemyXpos = self.enemyXpos + ((self.targetPosition - self.enemyXpos) * 0.08)

        ...

    def enemy_shoot(self, enemyXpos, enemyYpos, enemyTypeNum, enemyDirectionRight):

        currentWeapon = self.enemyGuns[enemyTypeNum]
        if enemyDirectionRight == True:
            bullet = self.__class__.Bullet(enemyXpos + currentWeapon.gunHolderLocalXpos + currentWeapon.barrelLocalXpos,
enemyYpos + currentWeapon.barrelLocalYpos, currentWeapon.shootingSpeed, currentWeapon.range, enemyDirectionRight,
currentWeapon.damage, 0)
        else:
            bullet = self.__class__.Bullet(enemyXpos - currentWeapon.gunHolderLocalXpos, enemyYpos +
currentWeapon.barrelLocalYpos, currentWeapon.shootingSpeed, currentWeapon.range, enemyDirectionRight,
currentWeapon.damage, 0)

        self.bullets.append(bullet)

```

Test 8 - Player's Score Increases Overtime

```

#SCORE
self.SCORE = max(self.SCORE - self.A_SCORE, int(self.playerProperties["Xpos"] - self.playerProperties["originalXpos"])) +
self.A_SCORE

self.flash_colour() #runs every second
text = "SCORE: " + str(self.SCORE)
self.display.text(text, 1, 1, scale=0.8)

...

if objectNum == 3:
    self.A_SCORE += ((object.enemyTypeNum + 1) * 100)

...

elif pickupTypeNum == 2:
    self.A_SCORE += 50
elif pickupTypeNum == 3:
    self.A_SCORE += 100
elif pickupTypeNum == 4:
    self.A_SCORE += 500

```

Test 9 - Player Health Updates Correctly

```

def heal(self):
    if self.healCounter < self.healRate:
        self.healCounter += 1
    else:
        self.HEALTH = min(100, self.HEALTH + 1)
        self.healCounter = 0
    ...

if pickupTypeNum == 0:
    self.HEALTH = min(100, self.HEALTH + 50)
elif pickupTypeNum == 1:
    self.HEALTH = 100
    ...

#HEALTH BAR
self.display.rectangle(self.WIDTH - 51, self.HEIGHT - 9, 50, 8)
self.display.set_pen(0)
self.display.rectangle(self.WIDTH - 50, self.HEIGHT - 8, 48, 6)
self.display.set_pen(15)
self.display.rectangle(self.WIDTH - 49, self.HEIGHT - 7, ((self.HEALTH) * 46) // 100, 4)

```

Test 10 - Help Pop Up panel

```

def help_menu(self):
    self.clear_screen()
    self.gfx.set_backlight(*self.Orange)
    self.display.set_pen(15)
    self.display.text("PIXEL RAMPAGE", 1, 1, scale=2.2)
    self.display.text("A - LEFT", 1, 17, scale=1)
    self.display.text("B - RIGHT", 1, 27, scale=1)
    self.display.text("C - SWITCH WEAPON->RESTART", 1, 37, scale=1)
    self.display.text("D - SHOOT", 1, 47, scale=1)
    self.display.text("E - JUMP->DOUBLE JUMP", 1, 57, scale=1)
    self.display.update()
    while True:
        if self.gfx.switch_pressed(SWITCH_A) or self.gfx.switch_pressed(SWITCH_B) or
self.gfx.switch_pressed(SWITCH_C) or self.gfx.switch_pressed(SWITCH_D) or self.gfx.switch_pressed(SWITCH_E):
            time.sleep(0.5)
            self.main_menu()

```

Test 11 - Player Animation

```
def change_player_frame(self):
    if (not self.gfx.switch_pressed(SWITCH_A) and not self.gfx.switch_pressed(SWITCH_B)) or not self.playerProperties["canMoveX"]:
        self.frameNum = 0
        return
    if self.frameCounter < self.frameLength:
        self.frameCounter += 1
    else:
        self.frameNum = (self.frameNum + 1) % len(self.playerFrames)
        self.frameCounter = 0
```

Test 12 - Navigation

```
def main_menu(self):
    #PIXEL RAMPAGE
    self.clear_screen()
    self.gfx.set_backlight(*self.Green)
    self.display.set_pen(15)
    self.display.text("PIXEL RAMPAGE", 1, 1, scale=2.2)
    self.display.text("A - PLAY", 3, 20, scale=1)
    self.display.text("B - LEADERBOARD", 54, 20, scale=1)
    self.display.text("C - HELP", 48, 30, scale=1)

    self.draw(self.enemyPixels4, 1, 43)
    self.draw(self.enemyPixels1, 30, 43)
    self.draw(self.enemyPixels1, 30, 54)
    self.draw(self.enemyPixels0, 46, 50)
    self.draw(self.playerFrame0, 59, 38)
    self.draw(self.guns[4].pixels, 67, 42)
    self.draw(self.enemyPixels0, 88, 50, 0, True)
    self.draw(self.enemyPixels2, 105, 39, 0, True)
    self.draw(self.enemyPixels2, 105, 52, 0, True)
    self.draw(self.enemyPixels3, 127, 47, 0, True)
    self.display.update()
    while True:
        if self.gfx.switch_pressed(SWITCH_A):
            time.sleep(0.5)
            self.run()
        elif self.gfx.switch_pressed(SWITCH_B):
            time.sleep(0.5)
            self.show_leaderboard()
        elif self.gfx.switch_pressed(SWITCH_C):
            time.sleep(0.5)
            self.help_menu()

def game_over_screen(self):
    self.clear_screen()
    self.gfx.set_backlight(*self.Green)

    #Update leaderboard if needed
    scoreText = "{:06}".format(min(999999, self.SCORE))
    onLeaderboard = self.update_leaderboard(scoreText)

    self.display.set_pen(15)
    self.display.text("GAME OVER", 20, 1, scale=2.2)
    self.display.text("A - RESTART", 3, 20, scale=1)
    self.display.text("B - MAIN MENU", 65, 20, scale=1)
    if onLeaderboard:
        self.display.text("your on the leaderboard!!!", 5, 31, scale=1)
    else:
        self.display.text("SCORE: ", 52, 31, scale=1)
    self.display.text(scoreText, 38, 40, scale=2)
```

```

self.draw(self.enemyTypes[4], 1, 43)
self.draw(self.guns[0].pixels, 33, 57)
self.draw(self.guns[1].pixels, 48, 57)
self.draw(self.guns[2].pixels, 63, 57)
self.draw(self.guns[3].pixels, 78, 57)
self.draw(self.guns[4].pixels, 93, 57)
self.draw(self.enemyTypes[3], 127, 47, 0, True)
self.display.update()
while True:
    if self.gfx.switch_pressed(SWITCH_A):
        #Restart
        time.sleep(0.5)
        self.__init__()
        self.run()
    elif self.gfx.switch_pressed(SWITCH_B):
        #Main Menu
        time.sleep(0.5)
        self.__init__()
        self.main_menu()

```

Test 13 - Restarting the Game

```

#RESTARTING
if self.gfx.switch_pressed(SWITCH_C):
    if self.restartCounter < self.restartCounterLength:
        self.restartCounter += 1
    else:
        self.playerRestart = True
else:
    self.restartCounter = 0

...

def is_Dead(self):
    if self.playerRestart:
        return True
    if (self.playerProperties["Ypos"] >= self.HEIGHT or self.HEALTH <= 0):
        #RESTART GAME
        return True
    return False

```

Test 14 - Leaderboard is Displayed and Updated Correctly

```

def show_leaderboard(self):
    self.clear_screen()
    self.gfx.set_backlight(*self.Red)
    self.display.set_pen(15)

    with open("leaderboard.txt", "r") as file:
        lines = file.readlines()
        lines = [line.strip() for line in lines]

    if lines == []:
        with open("leaderboard.txt", "w") as file:
            file.write("000000\n" * 10)
        lines = ["000000", "000000", "000000", "000000", "000000", "000000", "000000", "000000", "000000", "000000"]

    self.display.text("LEADERBOARD", 1, 1, scale=2.2)
    self.display.text("1. " + lines[0], 1, 17, scale=1)
    self.display.text("2. " + lines[1], 1, 27, scale=1)
    self.display.text("3. " + lines[2], 1, 37, scale=1)
    self.display.text("4. " + lines[3], 1, 47, scale=1)
    self.display.text("5. " + lines[4], 1, 57, scale=1)
    self.display.text("6. " + lines[5], 51, 17, scale=1)
    self.display.text("7. " + lines[6], 51, 27, scale=1)
    self.display.text("8. " + lines[7], 51, 37, scale=1)
    self.display.text("9. " + lines[8], 51, 47, scale=1)
    self.display.text("10. " + lines[9], 51, 57, scale=1)
    self.display.update()

    while True:
        if self.gfx.switch_pressed(SWITCH_A) or self.gfx.switch_pressed(SWITCH_B) or self.gfx.switch_pressed(SWITCH_C)
or self.gfx.switch_pressed(SWITCH_D) or self.gfx.switch_pressed(SWITCH_E):
            time.sleep(0.5)
            self.main_menu()

def update_leaderboard(self, newScore):
    with open("leaderboard.txt", "r") as file:
        lines = file.readlines()
        lines = [line.strip() for line in lines]

    if lines == []:
        with open("leaderboard.txt", "w") as file:
            file.write("000000\n" * 10)
        lines = ["000000", "000000", "000000", "000000", "000000", "000000", "000000", "000000", "000000", "000000"]

    if int(newScore) > int(lines[-1]):
        lines.pop()
        lines.append(newScore)
        lines.sort(key=int, reverse=True)
        lines = list(map("{:06}".format, lines))
        with open("leaderboard.txt", "w") as file:
            file.write("\n".join(lines))
        return True
    else:
        return False

```

Section 5 - Evaluation



Introduction

In this section I will reflect on the project by looking at how many specification points are fulfilled, identifying how the project may have been improved further and commenting on the overall quality of the finished product.

Comparing against Initial Objectives

As shown and proven in the testing section, I have achieved all my project's objectives, and here is my outlining how I have done this:

Objective	How was this achieved
Movement	The user can move left/rightjump/double jump with uniform acceleration curves to make movement smooth and effortless in all directions, with a constant gravitational force downwards
Shooting	In the final game, the user can switch between collected weapons and shoot them individually with their associated properties. Bullets are spawned from a specified position and after a certain amount of time
Scoring	Score is displayed correctly on the screen and increases with distance travelled and when certain pickups are collected
Health	It is shown via a health bar on the screen. The player will take damage when hit and also heal over time or when certain pickups are collected
Pickups	Pickups are spawned and fixed onto platforms which the user can interact with and use
Procedural generation of levels	The level will continue forever (aswell as storing previous parts and enemies), giving the player endless possibilities
Enemies	Enemies are spawned randomly on platforms and can move, shoot and take damage in game. There are a total of 5 different enemies with different characteristics
Animations	Player's walking/running is animated enhancing the retro feel and liveliness
Restart function	The player can restart the game at any point in time
User interface	The user can easily switch between the leaderboard, help, main menu and game screens
Ease of Use	The user can view a help screen which displays the controls
Leaderboard	Scores are saved locally and displayed on a leaderboard screen in a 6 digit format

How my solution may have been improved

Here are some basic ideas that I would have worked on if I had more time.

- Background - there is a lot of empty space when there are no enemies on screen, and the player is not shooting, so perhaps a skybox or certain props may have been appropriate to fill this space
- Wider variety of animations - including animations in enemies, explosions, pickup collection and weapon switching
- Further enemy functionality - allow enemies to move between platforms and chase the player or jump up

and down to avoid bullets. Although, the processing power of the pico is an acceptable limitation defined in my Analysis Section, if I had more time, I would have liked to experiment with reinforcement learning and neural networks to learn optimal actions reacting to gameplay.

- Using inheritance - I could have structured my code slightly differently by removing obsolete functions that are inherently similar between certain objects and creating classes that all inherit from one basic `game_object` class. However, I did optimise the structure a lot already in my current code, and it is unclear/optimistic how much of a benefit this approach would offer.
- Multiplayer Functionality - I think multiplayer games are much more fun as you can play with/against friends. This would have been an additional feature and used the pico W, which enables for wifi connectivity, and is upgraded hardware

Feedback from Mr Wilde

I showed my game to Mr Wilde (my end-user) during the development process and the final product when playing through it before offering for him to have a go. He seemed impressed and happy with the art style, gameplay, enemies and even the choice of name - Pixel Rampage.

Personal Reflection: Overall, I am satisfied with my product and enjoy playing it myself too. I have completed all my objectives outlined in the analysis, and gone further than what I originally planned to implement. I think my game is well connected, robust and a unique approach to a platformer. I hope that other's will like my game as much I did when making it.